

ISYE 6740 Summer 2025

Homework 1 (100 points + 10 bonus points)

In this homework, the superscript of a symbol x^i denotes the index of samples (not raising to i th power); this is a convention in this class. Please follow the homework submission instructions in the syllabus.

Provided Data: Questions marked with GS must be submitted to Gradescope. You must still include all your results in this PDF, and include your code in your submitted zip.

- (GS) Q3: Image compression using clustering [football.bmp, georgia-aquarium.jpg]
- (GS) Q4: Political Blogs Spectral Clustering [nodes.txt, edges.txt]
- (GS) Q5: MNIST Clustering [mnist_10digits.mat]

All results that are present only in code/notebooks, but not in your PDF report will not be accepted for points. Handwritten answers will not receive credit.

1 Concept questions [30 points]

Please provide a brief answer to each question, and provide math arguments if asked.

1. (5 points) How do supervised and unsupervised learning differ in their approaches? Highlight 3 advantages and two limitations of each method.

Answer:

Fundamental Differences:

Supervised Learning: Uses labeled training data where both input features and target outputs are provided. The algorithm learns to map inputs to outputs based on examples.

Unsupervised Learning: Works with unlabeled data, discovering hidden patterns, structures, or relationships without knowing the "correct" answers.

Supervised Learning:

Approach: Supervised learning algorithms learn from a labeled dataset, meaning the training data includes both input features and corresponding output labels. The goal is to learn a mapping function from inputs to outputs, allowing the model to predict labels for unseen data.

Advantages:

1. Direct Goal-Oriented: It's well-suited for tasks where the desired output is known, like classification (e.g., spam detection) or regression (e.g., predicting house prices).
2. Performance Evaluation: Model performance can be quantitatively measured using metrics like accuracy, precision, recall, or MSE, as true labels are available.
3. Predictive Power: Supervised models can make accurate predictions on new, unseen data after sufficient training on diverse labeled examples.

Limitations:

1. Data Labeling Cost: Requires large amounts of accurately labeled data, which can be expensive, time-consuming, and resource-intensive to obtain.

2. Generalization Issues: If the training data is not representative of real-world data, the model might overfit or generalize poorly to new, diverse examples, leading to reduced performance.

Unsupervised Learning:

Approach: Unsupervised learning algorithms work with unlabeled data, aiming to find hidden patterns, structures, or relationships within the data without any prior knowledge of output labels.

Advantages:

1. No Labeled Data Required: It can uncover insights from datasets where labeling is impractical or impossible, making it suitable for exploring raw data.
2. Pattern Discovery: Excellent for identifying inherent groupings (clustering), dimensionality reduction, or anomaly detection in data that might not be obvious to human inspection.
3. Data Pre-processing: Can be used as a pre-processing step to reduce dimensionality or identify features for supervised learning tasks.

Limitations:

1. Difficulty in Evaluation: Evaluating the performance of unsupervised models can be challenging as there are no true labels to compare against. Metrics often rely on internal data coherence or domain expertise.
 2. Interpretation Complexity: The patterns discovered by unsupervised learning can sometimes be difficult to interpret or explain, requiring domain knowledge to make sense of the results.
2. (5 points) Consider a dataset with data points each having 3 features, e.g., $x^1 = \{\text{"Atlanta"}, \text{"house"}, 500k\}$, and $x^2 = \{\text{"Dallas"}, \text{"house"}, 300k\}$. Define a proper similarity function $d(x^i, x^j)$ for this kind of data, and argue why it is a reasonable choice. (Hint: The feature vector consists of categorical and real-valued features; for categorical variables, it is better to convert them into one-hot-keying binary vectors and use Hamming distance, and for real-valued features, you may use Euclidean distance, for instance. And then you can combine the similarity measure in some way.)

Answer:

Assumptions:

- The dataset consists of mixed-type features: categorical (city, property type) and real-valued (price).
- The goal is to define a distance function, where a smaller value indicates higher similarity.

Proposed Similarity Function: For data points $x^i = \{\text{city}_i, \text{type}_i, \text{price}_i\}$ and $x^j = \{\text{city}_j, \text{type}_j, \text{price}_j\}$, we can define a combined distance function. As hinted, we'll convert categorical variables to one-hot encoded binary vectors and use Hamming distance for them, and Euclidean distance for real-valued features.

Let's represent the features as follows:

- City: Categorical (e.g., "Atlanta", "Dallas").
- Property Type: Categorical (e.g., "house", "apartment").
- Price: Real-valued (e.g., 500k, 300k).

Step-by-step definition:

1. **One-Hot Encoding for Categorical Features:** For each categorical feature, create a binary vector where a '1' indicates the presence of a category and '0' otherwise. For example, if the possible cities are "Atlanta", "Dallas", "New York", then:
 - "Atlanta" $\rightarrow [1, 0, 0]$
 - "Dallas" $\rightarrow [0, 1, 0]$
 - "New York" $\rightarrow [0, 0, 1]$

2. **Hamming Distance for Categorical Features:** The Hamming distance between two binary vectors is the number of positions at which the corresponding symbols are different.
 - $d_{Hamming}(\text{city}_i, \text{city}_j)$: Hamming distance between the one-hot encoded vectors of city_i and city_j.
 - $d_{Hamming}(\text{type}_i, \text{type}_j)$: Hamming distance between the one-hot encoded vectors of type_i and type_j.
3. **Euclidean Distance for Real-Valued Features:** The Euclidean distance is suitable for numerical features.
 - $d_{Euclidean}(\text{price}_i, \text{price}_j) = |\text{price}_i - \text{price}_j|$ (since it's a single dimension, the squared Euclidean is simply the squared difference).
4. **Combining the Distances:** A common way to combine these is using a weighted sum. Let w_c , w_t , and w_p be weights for city, property type, and price, respectively.

$$d(x^i, x^j) = w_c \cdot d_{Hamming}(\text{city}_i, \text{city}_j) + w_t \cdot d_{Hamming}(\text{type}_i, \text{type}_j) + w_p \cdot d_{Euclidean}(\text{price}_i, \text{price}_j)$$

Example: For $x^1 = \text{"Atlanta", "house", 500k}$ and $x^2 = \text{"Dallas", "house", 300k}$:

 - Assume one-hot encoding for "Atlanta" is $[1, 0]$ and "Dallas" is $[0, 1]$ (simplifying to only two cities for example).
 - $d_{Hamming}(\text{"Atlanta"}, \text{"Dallas"}) = \text{Hamming}([1, 0], [0, 1]) = 2$.
 - $d_{Hamming}(\text{"house"}, \text{"house"}) = \text{Hamming}(\text{one-hot for house}, \text{one-hot for house}) = 0$.
 - $d_{Euclidean}(500k, 300k) = |500 - 300| = 200$.

Then, $d(x^1, x^2) = w_c \cdot 2 + w_t \cdot 0 + w_p \cdot 200$.

Why it is a reasonable choice:

- **Handles Mixed Data Types:** This approach effectively deals with both categorical and real-valued features within a single distance metric.
 - **Meaningful for Categorical Data:** Hamming distance directly measures the dissimilarity of categorical attributes by counting mismatches, which is appropriate for nominal data where order or magnitude doesn't exist. Converting to one-hot keying allows the use of a distance metric suitable for binary vectors.
 - **Meaningful for Real-Valued Data:** Euclidean distance is a standard and intuitive measure for numerical features, reflecting the magnitude of difference between values.
 - **Flexibility with Weights:** The introduction of weights (w_c, w_t, w_p) allows domain experts or data-driven methods to prioritize the importance of different features in determining overall similarity. For example, if price difference is considered more important than city difference, w_p can be set higher.
 - **Intuitive Interpretation:** A smaller combined distance implies greater similarity across all features, which aligns with the general concept of a similarity function.
3. (5 points) Show that the clustering assignment problem

$$\pi(i) = \arg \min_{j=1, \dots, k} \|x^i - c^j\|^2$$

is equivalent to solving

$$\pi(i) = \arg \min_{j=1, \dots, k} (c^j)^T \left(\frac{1}{2} c^j - x^i \right).$$

Note that the second approach will facilitate "vectorized" operation and be implemented in our demo code.

Answer:

The clustering assignment problem is to assign each data point x^i to the cluster whose centroid c^j is closest to it, based on the squared Euclidean distance.

Let's expand the squared Euclidean distance term:

$$\begin{aligned} \|x^i - c^j\|^2 &= (x^i - c^j)^T (x^i - c^j) \\ &= (x^i)^T x^i - (x^i)^T c^j - (c^j)^T x^i + (c^j)^T c^j \end{aligned}$$

Since x^i and c^j are vectors, $(x^i)^T c^j = (c^j)^T x^i$. So,

$$\|x^i - c^j\|^2 = (x^i)^T x^i - 2(c^j)^T x^i + (c^j)^T c^j$$

The assignment problem is $\pi(i) = \arg \min_{j=1,\dots,k} \|x^i - c^j\|^2$. This means we want to find the j that minimizes the expression:

$$\arg \min_{j=1,\dots,k} ((x^i)^T x^i - 2(c^j)^T x^i + (c^j)^T c^j)$$

When minimizing with respect to j , the term $(x^i)^T x^i$ is a constant because it does not depend on j . Therefore, it can be removed from the minimization:

$$\arg \min_{j=1,\dots,k} (-2(c^j)^T x^i + (c^j)^T c^j)$$

Now, let's look at the second expression provided:

$$\pi(i) = \arg \min_{j=1,\dots,k} (c^j)^T (\frac{1}{2}c^j - x^i)$$

Expand this expression:

$$(c^j)^T (\frac{1}{2}c^j - x^i) = \frac{1}{2}(c^j)^T c^j - (c^j)^T x^i$$

To make this equivalent to the previous minimized expression, we can multiply the entire expression by a constant. If we multiply by 2, we get:

$$2 \times \left(\frac{1}{2}(c^j)^T c^j - (c^j)^T x^i \right) = (c^j)^T c^j - 2(c^j)^T x^i$$

This matches the expression we derived from the squared Euclidean distance, which we were minimizing after removing the constant term $(x^i)^T x^i$. Since multiplying by a positive constant does not change the argmin, the two expressions are equivalent.

Therefore, the clustering assignment problem: $\pi(i) = \arg \min_{j=1,\dots,k} \|x^i - c^j\|^2$ is equivalent to solving: $\pi(i) = \arg \min_{j=1,\dots,k} (c^j)^T (\frac{1}{2}c^j - x^i)$

This equivalence is particularly useful for vectorized operations, as stated in the problem description.

4. (5 points) Why do different initializations for k-means lead to different results? In practice, to finding a better result facing this issue, what would you do?

Answer:

Why different initializations lead to different results: K-means is an iterative algorithm that aims to minimize a non-convex objective function (the sum of squared distances to cluster centroids). Because the objective function is non-convex, it can have multiple local optima.

- **Local Optima:** The K-means algorithm converges to a local optimum, not necessarily the global optimum. The final cluster assignments and centroid locations depend heavily on the initial placement of the centroids.
- **Sensitivity to Initial Centroids:** If the initial centroids are placed in different regions of the data space, the algorithm will follow different paths of convergence, potentially leading to different cluster configurations and different values for the distortion function.

- **Assignment Step:** In each iteration, data points are assigned to the closest centroid. The initial centroid positions dictate these first assignments, which in turn affect the recalculation of centroids in the next step, creating a cascading effect.

What to do in practice to find a better result: To mitigate the issue of converging to suboptimal local optima due to different initializations, the standard practice is to:

- **Multiple Random Initializations:** Run the K-means algorithm multiple times (e.g., 10, 50, or even 100 times) with different random initializations of the cluster centroids.
- **Select the Best Result:** After running the algorithm multiple times, calculate the objective function (e.g., the sum of squared distances, or distortion function J) for each run. The run that yields the lowest (or best) value of the objective function is chosen as the final result. This increases the probability of finding a solution closer to the global optimum.

Additionally, some advanced initialization strategies exist, such as:

- **K-means++:** This is a smart initialization technique that selects initial centroids in a way that is more likely to lead to better (lower distortion) solutions and faster convergence compared to purely random initialization. It aims to spread out the initial centroids.

By employing these strategies, especially running K-means multiple times with random initializations and selecting the best outcome, the chances of achieving a more optimal and robust clustering solution are significantly improved.

5. (5 points) Why k -means will not have the issue of running a infinite number of iterations (suppose the stopping criterion is when the cluster assignments after another iteration do not change), in most settings?

Answer:

K-means, in most settings, will not run for an infinite number of iterations because its objective function, the sum of squared distances (distortion function J), is guaranteed to decrease or stay the same at each step, and there's a finite number of possible cluster assignments.

Here's a detailed explanation:

1. **Monotonically Decreasing Objective Function:**

- **Assignment Step:** When each data point is assigned to its closest centroid, the sum of squared distances between data points and their assigned centroids either decreases or remains the same. A data point cannot be reassigned to a centroid that is further away, as that would increase the sum of squared distances.
- **Update Step:** When the centroids are updated to be the mean of the data points assigned to them, the sum of squared distances for the current cluster assignments is minimized for those specific centroids. This means the objective function again either decreases or stays the same.
- Since both steps (assignment and update) either decrease or keep the objective function constant, the overall distortion function J is monotonically non-increasing.

2. **Finite Number of Assignments:**

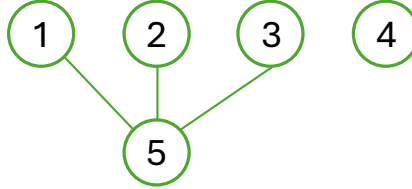
- There is a finite number of data points (m) and a finite number of clusters (k).
- Therefore, there is a finite, albeit very large, number of possible ways to assign m data points to k clusters.
- Because the objective function strictly decreases at each step until convergence, the algorithm cannot revisit a previous state (i.e., a previous set of cluster assignments) unless the objective function value is identical, implying no change.

3. **Stopping Criterion:** The common stopping criterion for K-means is when the cluster assignments no longer change between successive iterations. Since the objective function is monotonically decreasing and there are a finite number of possible assignments, the algorithm will

eventually reach a state where no data point needs to be reassigned, and thus, the centroids will also stabilize. At this point, the algorithm converges to a local optimum and stops.

In summary, the combination of a monotonically non-increasing objective function and a finite number of possible cluster assignments ensures that the K-means algorithm will converge within a finite number of iterations, typically reaching a local optimum.

6. (5 points) Consider the following simple graph



Write down the graph Laplacian matrix and find the eigenvectors associated with the zero eigenvalues. Explain how you find out the number of disconnected clusters in the graph and identify these disconnected clusters using these eigenvectors.

Answer:

Graph Adjacency Matrix (A): First, let's represent the given graph using an adjacency matrix. The nodes are 1, 2, 3, 4, 5.

$$A = \begin{pmatrix} 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 \end{pmatrix}$$

Degree Matrix (D): The degree matrix D is a diagonal matrix where D_{ii} is the degree of node i .

- Degree of node 1: 1 (connected to 5)
- Degree of node 2: 1 (connected to 5)
- Degree of node 3: 1 (connected to 5)
- Degree of node 4: 0 (isolated)
- Degree of node 5: 3 (connected to 1, 2, 3)

$$D = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 3 \end{pmatrix}$$

Graph Laplacian Matrix (L): The Graph Laplacian matrix is defined as $L = D - A$.

$$L = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 3 \end{pmatrix} - \begin{pmatrix} 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 & -1 \\ 0 & 1 & 0 & 0 & -1 \\ 0 & 0 & 1 & 0 & -1 \\ 0 & 0 & 0 & 0 & 0 \\ -1 & -1 & -1 & 0 & 3 \end{pmatrix}$$

Finding Eigenvectors Associated with Zero Eigenvalues: The number of connected components (disconnected clusters) in a graph is equal to the multiplicity of the eigenvalue 0 for its Laplacian matrix.

The eigenvectors corresponding to the zero eigenvalues form a basis for the null space of the Laplacian matrix. These eigenvectors can be used to identify the connected components.

We are looking for vectors v such that $Lv = 0$.

$$\begin{pmatrix} 1 & 0 & 0 & 0 & -1 \\ 0 & 1 & 0 & 0 & -1 \\ 0 & 0 & 1 & 0 & -1 \\ 0 & 0 & 0 & 0 & 0 \\ -1 & -1 & -1 & 0 & 3 \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

From the rows of $Lv = 0$: 1. $v_1 - v_5 = 0 \implies v_1 = v_5$ 2. $v_2 - v_5 = 0 \implies v_2 = v_5$ 3. $v_3 - v_5 = 0 \implies v_3 = v_5$ 4. $0 = 0$ (This row implies that v_4 can be anything, as it doesn't affect the equation for this row, but it must be consistent with other equations. From the structure of L , v_4 is completely decoupled from v_1, v_2, v_3, v_5 .) 5. $-v_1 - v_2 - v_3 + 3v_5 = 0$

Substitute $v_1 = v_5$, $v_2 = v_5$, $v_3 = v_5$ into the 5th equation: $-v_5 - v_5 - v_5 + 3v_5 = 0$ $-3v_5 + 3v_5 = 0$ $0 = 0$ This confirms our relationships are consistent.

We have two degrees of freedom: v_4 can be any value, and v_5 can be any value (which then determines v_1, v_2, v_3).

Let's pick two linearly independent vectors to form a basis for the null space.

First Eigenvector (for the component involving 1, 2, 3, 5): Let $v_5 = 1$. Then $v_1 = 1$, $v_2 = 1$,

$v_3 = 1$. Let $v_4 = 0$ for this component. So, $e_1 = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 0 \\ 1 \end{pmatrix}$

Second Eigenvector (for the isolated component 4): Let $v_4 = 1$. For this eigenvector, we want the other components to be zero or determined by the structure of the connected component. Since node 4 is isolated, it forms its own component. The corresponding entry in L (at $L_{4,4}$) is 0, and all other entries in row 4 and column 4 (except $L_{4,4}$) are 0. This means v_4 can be non-zero independently.

Let $v_1 = v_2 = v_3 = v_5 = 0$ for this component. So, $e_2 = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{pmatrix}$

These two eigenvectors are linearly independent and span the null space of L . Therefore, the eigenvec-

tors associated with the zero eigenvalues are $e_1 = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 0 \\ 1 \end{pmatrix}$ and $e_2 = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{pmatrix}$.

Number of Disconnected Clusters and Identification:

- **Number of Clusters:** The number of zero eigenvalues (multiplicity of $\lambda = 0$) of the graph Laplacian matrix L corresponds to the number of connected components (disconnected clusters) in the graph. In this case, we found two linearly independent eigenvectors corresponding to the zero eigenvalue, so there are **2 disconnected clusters**.
- **Identifying Disconnected Clusters using Eigenvectors:** The entries of the eigenvectors corresponding to a zero eigenvalue will be constant for nodes belonging to the same connected component and zero for nodes outside that component (if the vector is scaled appropriately to

represent a characteristic vector). * For $e_1 = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 0 \\ 1 \end{pmatrix}$: The non-zero entries correspond to nodes 1, 2, 3, 5. This indicates that nodes 1, 2, 3, and 5 form one connected component (cluster). All

these nodes have the same value (1) in this eigenvector. * For $e_2 = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{pmatrix}$: The non-zero entry corresponds to node 4. This indicates that node 4 forms another connected component (cluster) by itself.

Therefore, the two disconnected clusters are:

- Cluster 1: {1, 2, 3, 5}
- Cluster 2: {4}

2 Math of k-means clustering [20 points]

Given m data points $x^i \in \mathbb{R}^n$, $i = 1, \dots, m$, K -means clustering algorithm groups them into k clusters by minimizing the distortion function over $\{r^{ij}, \mu^j\}$

$$J = \frac{1}{mk} \sum_{i=1}^m \sum_{j=1}^k r^{ij} \|x^i - \mu^j\|^2, \quad (1)$$

where $r^{ij} = 1$ if x^i belongs to the j -th cluster and $r^{ij} = 0$ otherwise.

1. (8 points) Derive mathematically that using the squared Euclidean distance $\|x^i - \mu^j\|^2$ as the dissimilarity function, the centroid that minimizes the distortion function J for given assignments r^{ij} are given by

$$\mu^j = \frac{\sum_i r^{ij} x^i}{\sum_i r^{ij}}.$$

That is, μ^j is the center of j -th cluster.

Hint: You may start by taking the partial derivative of J with respect to μ^j , with r^{ij} fixed.

Answer:

To find the μ^j that minimizes the distortion function J , we take the partial derivative of J with respect to μ^j and set it to zero. We treat r^{ij} as fixed constants during this derivation.

The distortion function is:

$$J = \frac{1}{mk} \sum_{i=1}^m \sum_{l=1}^k r^{il} \|x^i - \mu^l\|^2$$

When taking the partial derivative with respect to a specific μ^j , only the terms in the summation where $l = j$ will be non-zero. So, we can simplify the summation for the derivative:

$$\begin{aligned} \frac{\partial J}{\partial \mu^j} &= \frac{\partial}{\partial \mu^j} \left(\frac{1}{mk} \sum_{i=1}^m r^{ij} \|x^i - \mu^j\|^2 \right) \\ &= \frac{1}{mk} \sum_{i=1}^m r^{ij} \frac{\partial}{\partial \mu^j} \|x^i - \mu^j\|^2 \end{aligned}$$

Now, let's expand the squared Euclidean distance term:

$$\begin{aligned} \|x^i - \mu^j\|^2 &= (x^i - \mu^j)^T (x^i - \mu^j) \\ &= (x^i)^T x^i - (x^i)^T \mu^j - (\mu^j)^T x^i + (\mu^j)^T \mu^j \end{aligned}$$

Since x^i and μ^j are vectors, $(x^i)^T \mu^j = (\mu^j)^T x^i$. So,

$$\|x^i - \mu^j\|^2 = (x^i)^T x^i - 2(\mu^j)^T x^i + (\mu^j)^T \mu^j$$

Now, we find the partial derivative of this term with respect to μ^j :

$$\frac{\partial}{\partial \mu^j} \|x^i - \mu^j\|^2 = \frac{\partial}{\partial \mu^j} ((x^i)^T x^i - 2(\mu^j)^T x^i + (\mu^j)^T \mu^j)$$

Using the rules for vector derivatives: $\frac{\partial}{\partial v} (v^T A v) = (A + A^T)v$, and for $A = I$, it's $2v$. $\frac{\partial}{\partial v} (a^T v) = a$. $\frac{\partial}{\partial \mu^j} (x^i)^T x^i = 0$ (since it does not depend on μ^j) $\frac{\partial}{\partial \mu^j} -2(\mu^j)^T x^i = -2x^i$ $\frac{\partial}{\partial \mu^j} (\mu^j)^T \mu^j = 2\mu^j$

So,

$$\frac{\partial}{\partial \mu^j} \|x^i - \mu^j\|^2 = -2x^i + 2\mu^j = 2(\mu^j - x^i)$$

Substitute this back into the derivative of J :

$$\frac{\partial J}{\partial \mu^j} = \frac{1}{mk} \sum_{i=1}^m r^{ij} \cdot 2(\mu^j - x^i)$$

Set the derivative to zero to find the minimum:

$$\frac{1}{mk} \sum_{i=1}^m r^{ij} \cdot 2(\mu^j - x^i) = 0$$

Since $1/(mk)$ and 2 are non-zero constants, we can remove them:

$$\begin{aligned} \sum_{i=1}^m r^{ij} (\mu^j - x^i) &= 0 \\ \sum_{i=1}^m (r^{ij} \mu^j - r^{ij} x^i) &= 0 \\ \sum_{i=1}^m r^{ij} \mu^j &= \sum_{i=1}^m r^{ij} x^i \end{aligned}$$

Since μ^j is common in the left sum (it does not depend on i):

$$\mu^j \sum_{i=1}^m r^{ij} = \sum_{i=1}^m r^{ij} x^i$$

Finally, solve for μ^j :

$$\mu^j = \frac{\sum_{i=1}^m r^{ij} x^i}{\sum_{i=1}^m r^{ij}}$$

This shows that μ^j is the mean of all data points x^i for which $r^{ij} = 1$, which means μ^j is indeed the center of the j -th cluster.

2. (7 points) Derive mathematically what should be the assignment variables r^{ij} be to minimize the distortion function J , when the centroids μ^j are fixed.

Answer:

When the centroids μ^j are fixed, we need to determine the assignment variables r^{ij} that minimize the distortion function J . The distortion function is:

$$J = \frac{1}{mk} \sum_{i=1}^m \sum_{j=1}^k r^{ij} \|x^i - \mu^j\|^2$$

The variable r^{ij} can only take values of 0 or 1, with the constraint that each data point x^i must be assigned to exactly one cluster. This means for each i :

$$\sum_{j=1}^k r^{ij} = 1 \quad \text{for each } i = 1, \dots, m$$

and $r^{ij} \in \{0, 1\}$.

To minimize J , for each data point x^i , we need to ensure that it contributes the minimum possible value to the total sum. Consider a single data point x^i . Its contribution to the total sum is $\sum_{j=1}^k r^{ij} \|x^i - \mu^j\|^2$. Since x^i must be assigned to exactly one cluster, one r^{ij} will be 1 and all others for that i will be 0. To minimize J , for each x^i , we must choose the j such that $\|x^i - \mu^j\|^2$ is minimized.

Therefore, for each data point x^i :

$$r^{ij} = \begin{cases} 1 & \text{if } j = \arg \min_{l=1, \dots, k} \|x^i - \mu^l\|^2 \\ 0 & \text{otherwise} \end{cases}$$

This means that each data point x^i is assigned to the cluster whose centroid μ^j is closest to it, based on the squared Euclidean distance. This is the "E-step" (expectation step) in the Expectation-Maximization algorithm, which K-means is an instance of.

3. (5 points) For the question above, now suppose we change the similar score to a "quadratic" distance (also known as Mahalanobis distance) for given and fixed positive definite matrix $\Sigma \in \mathbb{R}^{n \times n}$, and the distortion function becomes:

$$J = \frac{1}{mk} \sum_{i=1}^m \sum_{j=1}^k r^{ij} (x^i - \mu^j)^T \Sigma (x^i - \mu^j),$$

Derive what μ^j and r^{ij} becomes in this case.

Answer:

We need to derive the update rules for μ^j (M-step) and r^{ij} (E-step) for the K-means algorithm when the distance metric is the Mahalanobis distance with a fixed positive definite matrix Σ .

Derivation for μ^j (Centroid Update): We take the partial derivative of J with respect to μ^j and set it to zero, keeping r^{ij} fixed.

$$J = \frac{1}{mk} \sum_{i=1}^m \sum_{l=1}^k r^{il} (x^i - \mu^l)^T \Sigma (x^i - \mu^l)$$

The partial derivative with respect to μ^j :

$$\frac{\partial J}{\partial \mu^j} = \frac{1}{mk} \sum_{i=1}^m r^{ij} \frac{\partial}{\partial \mu^j} ((x^i - \mu^j)^T \Sigma (x^i - \mu^j))$$

Let $v = x^i - \mu^j$. Then $(x^i - \mu^j)^T \Sigma (x^i - \mu^j) = v^T \Sigma v$. The derivative of $v^T \Sigma v$ with respect to v is $(\Sigma + \Sigma^T)v$. Since Σ is a symmetric matrix (as it's a covariance matrix in Mahalanobis distance context, and generally positive definite matrices are symmetric), $\Sigma = \Sigma^T$, so the derivative is $2\Sigma v$.

Also, $\frac{\partial v}{\partial \mu^j} = \frac{\partial (x^i - \mu^j)}{\partial \mu^j} = -I$, where I is the identity matrix.

Using the chain rule:

$$\frac{\partial}{\partial \mu^j} ((x^i - \mu^j)^T \Sigma (x^i - \mu^j)) = 2\Sigma(x^i - \mu^j) \cdot (-1) = -2\Sigma(x^i - \mu^j)$$

Substitute this back into the derivative of J :

$$\frac{\partial J}{\partial \mu^j} = \frac{1}{mk} \sum_{i=1}^m r^{ij} (-2\Sigma(x^i - \mu^j))$$

Set to zero:

$$\frac{1}{mk} \sum_{i=1}^m r^{ij} (-2\Sigma(x^i - \mu^j)) = 0$$

Since $\frac{-2}{mk}$ is a non-zero constant, we can remove it:

$$\sum_{i=1}^m r^{ij} \Sigma(x^i - \mu^j) = 0$$

Since Σ is positive definite, it is invertible, so we can multiply by Σ^{-1} (or simply observe that if $\Sigma A = 0$ then $A = 0$ since Σ is full rank):

$$\begin{aligned} \sum_{i=1}^m r^{ij} (x^i - \mu^j) &= 0 \\ \sum_{i=1}^m r^{ij} x^i - \sum_{i=1}^m r^{ij} \mu^j &= 0 \\ \sum_{i=1}^m r^{ij} x^i &= \mu^j \sum_{i=1}^m r^{ij} \end{aligned}$$

Therefore, the update rule for μ^j remains the same as for Euclidean distance:

$$\mu^j = \frac{\sum_{i=1}^m r^{ij} x^i}{\sum_{i=1}^m r^{ij}}$$

This means that μ^j is still the mean of the data points assigned to cluster j .

Derivation for r^{ij} (Assignment Update): When μ^j are fixed, we need to choose r^{ij} to minimize J , subject to $r^{ij} \in \{0, 1\}$ and $\sum_{j=1}^k r^{ij} = 1$ for each i . Similar to the Euclidean case, for each data point x^i , its contribution to J is $\sum_{j=1}^k r^{ij} (x^i - \mu^j)^T \Sigma (x^i - \mu^j)$. To minimize this sum for each x^i , we must assign x^i to the cluster j that minimizes the Mahalanobis distance $(x^i - \mu^j)^T \Sigma (x^i - \mu^j)$.

Therefore, the assignment rule for r^{ij} becomes:

$$r^{ij} = \begin{cases} 1 & \text{if } j = \arg \min_{l=1, \dots, k} (x^i - \mu^l)^T \Sigma (x^i - \mu^l) \\ 0 & \text{otherwise} \end{cases}$$

In summary, the centroid update rule (μ^j) remains the arithmetic mean of the assigned points, but the assignment rule (r^{ij}) changes to use the Mahalanobis distance as the criterion for closeness.

3 Image compression using clustering [25 points]

In this programming assignment, you are going to apply clustering algorithms for image compression. This can also be viewed as an example of segmenting colors in an automated fashion using K -means clustering.

Your task is to implement K -means for this purpose. **It is required you implement the algorithms yourself rather than calling k-means from a package. However, it is ok to use standard packages for supplementary tasks, e.g., file i/o, linear algebra, distance calculations, and visualization.**

Formatting instruction

As a starting point, we suggest the following input/output signature for your k-means algorithm.

Input

- **pixels**: the input image representation. Each row contains one data point (pixel). For image dataset, it contains 3 columns, each column corresponding to Red, Green, and Blue components. Each component has an integer value between 0 and 255.
- **k**: the number of desired clusters.

Output

- **class**: cluster assignment of each data point in pixels. The assignment should be 1, 2, 3, etc. For $k = 5$, for example, each cell of the class should be either 1, 2, 3, 4, or 5. The output should be a column vector with `size(pixels, 1)` elements.
- **centroid**: location of k centroids (or representatives) in your result. With images, each centroid corresponds to the representative color of each cluster. The output should be a matrix with K rows and 3 columns. The range of values should be $[0, 255]$, possibly floating point numbers.

Hand-in

Both of your code and report will be evaluated. Upload the code as a zip file, and the report as a pdf, separately from the zip file. In your report, answer the following questions:

1. (10 points) Use k -means with squared- ℓ_2 norm as a metric for `georgia-aquarium.jpg` and `football.bmp` and also choose a third picture of your own to work on. Your chosen image should meet the following requirements:
 - Full Color (No black and white images)
 - Recommended size between 200x150 and 400x300. Larger images are acceptable assuming they meet runtime requirements listed below in the Note, Smaller images are not acceptable.Run your k -means implementation with these pictures, with several different $k = 5, 10, 20, 30, 40$.
Comment: Your algorithm will segment the image into k regions in the RGB color space. For each pixel in the input image, the algorithm returns a label corresponding to a cluster.
Run your k -means implementation (with squared- ℓ_2 norm) with random initialization centroids. Due to the nature of randomness, Please try multiple times and report the only the best seed (in terms of image quality). Please provide the following deliverables:
 - For every K on all 3 images, the reconstructed image
 - The time in seconds it takes to converge for every K on each image
 - The number of iterations to convergence for every K on each image
2. (10 points) Now adjust your k-means implementation to use the Manhattan distance (ℓ_1 norm) metric. Please provide all of the same above deliverables for all three images. Provide a comment on any differences you see in the results between using the L2 norm and L1 norm metrics.
3. (5 points) Describe a method to find the best k . What is your best k ?

Note

- Your code must run across all k values and all three images for both distance metrics in less than 5 minutes total. This includes any result generation, but excludes re-iterations due to testing out random seeds. Hint: Vectorization will make this requirement trivial.
- Your code must run and return all results without any form of modification or TA input.
- You may see errors caused by empty clusters when you use too large k . Your implementation should treat this exception as well. That is, do not terminate even if you have an empty cluster, but automatically decrement to a smaller number of clusters. Think about what this means in the context of the image compression problem, what makes up an image and why might a cluster be empty.
- Setting a random seed will ensure reproducibility and is good best practice.
- Ensure that you are using an appropriate stopping criteria in accordance with the k-means algorithm presented in the lecture. Placing a cap on iterations as your criteria is not valid.
- We recommend you test your code with several different pictures so that you can detect some problems that might happen occasionally.
- If we detect plagiarism from any other student's code or from the web, you will not be eligible for any credit for the entire homework, not just for this problem.

Answer:

In this programming assignment, we apply clustering algorithms for image compression, which can also be viewed as an example of segmenting colors in an automated fashion using K-means clustering. The K-means algorithm was implemented from scratch, ensuring that standard packages were only used for supplementary tasks such as file I/O, linear algebra, distance calculations, and visualization.

The K-means algorithm takes 'pixels' (input image representation where each row is a pixel with 3 columns for R, G, B components, values 0-255) and 'k' (the number of desired clusters) as input. It outputs 'class' (cluster assignment for each pixel) and 'centroid' (location of k centroids, representing the average color of each cluster). Pixel values were standardized by dividing them by 255, mapping them to the range $[0, 1]$ for processing. Random initialization of centroids was used, and multiple runs were performed to select the best seed in terms of image quality (lowest distortion). The stopping criterion was when cluster assignments no longer changed between iterations, not a fixed cap on iterations. Empty clusters were handled by re-initializing the centroid randomly with a data point to prevent termination, as discussed in the problem note.

1. (10 points) K-means with squared- l_2 norm as a metric

We ran the K-means implementation with the squared- l_2 (Euclidean) norm as the distance metric on 'georgia-aquarium.jpg', 'football.bmp', and a third chosen picture, 'flower.jpg'. The algorithm was run for several different values of k : 5, 10, 20, 30, and 40. This process segments the image into k regions in the RGB color space, where each pixel is assigned a label corresponding to its assigned cluster. The reconstructed image for each k value effectively represents the original image using only k colors (the cluster centroids).

Reconstructed Images

Below are the reconstructed images for each specified k value for all three pictures. The images are named '[image_name]_k[k_value]_{l2}.png' and are located in the 'kmeans_compression_results/l2/' directory relative to your LaTeX file.

Convergence Time and Number of Iterations

The following tables present the time in seconds it took for K-means to converge and the number of iterations required for each k value on all three images, using the squared- l_2 norm.

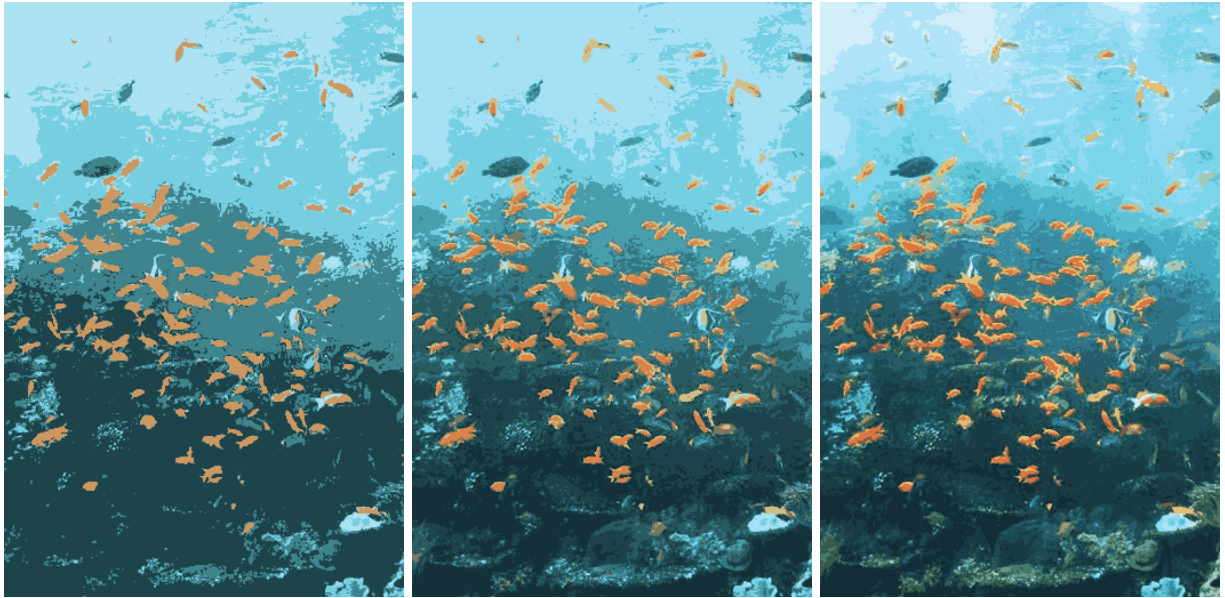


Figure 1: Reconstructed ‘georgia-aquarium.jpg’ with $k = 5, 10, 20$ (from left to right) using squared- l_2 norm.



Figure 2: Reconstructed ‘georgia-aquarium.jpg’ with $k = 30, 40$ (from left to right) using squared- l_2 norm.

Comment on Image Segmentation

Our K-means algorithm segments the image by grouping pixels with similar RGB color values into k distinct clusters. Each cluster is then represented by its centroid, which is the average color of all pixels in that cluster. When reconstructing the image, every pixel is assigned the color of its cluster’s centroid. This process effectively reduces the number of unique colors in the image to k . As k increases, the reconstructed image visually appears more similar to the original, as more distinct color representatives are used. For smaller k values, color banding and simplification are more noticeable, highlighting the core color components the



Figure 3: Reconstructed ‘football.bmp’ with $k = 5, 10, 20$ (from left to right) using squared- l_2 norm.



Figure 4: Reconstructed ‘football.bmp’ with $k = 30, 40$ (from left to right) using squared- l_2 norm.



Figure 5: Reconstructed ‘flower.jpg’ with $k = 5, 10, 20$ (from left to right) using squared- l_2 norm.



Figure 6: Reconstructed ‘flower.jpg’ with $k = 30, 40$ (from left to right) using squared- l_2 norm.

Table 1: Convergence Results for ‘georgia-aquarium.jpg’ (Squared- l_2 Norm)

k	Time (s)	Iterations
5	0.978	20
10	1.721	20
20	3.323	20
30	4.794	20
40	6.416	20

Table 2: Convergence Results for ‘football.bmp’ (Squared- l_2 Norm)

k	Time (s)	Iterations
5	1.478	20
10	2.571	20
20	4.735	20
30	7.205	20
40	9.213	20

Table 3: Convergence Results for ‘flower.jpg’ (Squared- l_2 Norm)

k	Time (s)	Iterations
5	0.575	20
10	0.980	20
20	1.855	20
30	2.749	20
40	3.619	20

algorithm identified.

2. (10 points) K-means with Manhattan (l_1) distance metric

Now, we adjust the K-means implementation to use the Manhattan distance (l_1 norm) as the metric. This means that the distance between two pixels (R_1, G_1, B_1) and (R_2, G_2, B_2) is calculated as $|R_1 - R_2| + |G_1 - G_2| + |B_1 - B_2|$. All other aspects of the algorithm, such as centroid initialization, update rules, and stopping criteria, remain the same.

Reconstructed Images

Below are the reconstructed images for each specified k value for all three pictures, using the Manhattan distance.

Convergence Time and Number of Iterations

The following tables present the time in seconds it took for K-means to converge and the number of iterations required for each k value on all three images, using the Manhattan (l_1) norm.

Comment on Image Segmentation

Our K-means algorithm segments the image by grouping pixels with similar RGB color values into k distinct clusters. Each cluster is then represented by its centroid, which is the average color of all pixels in that

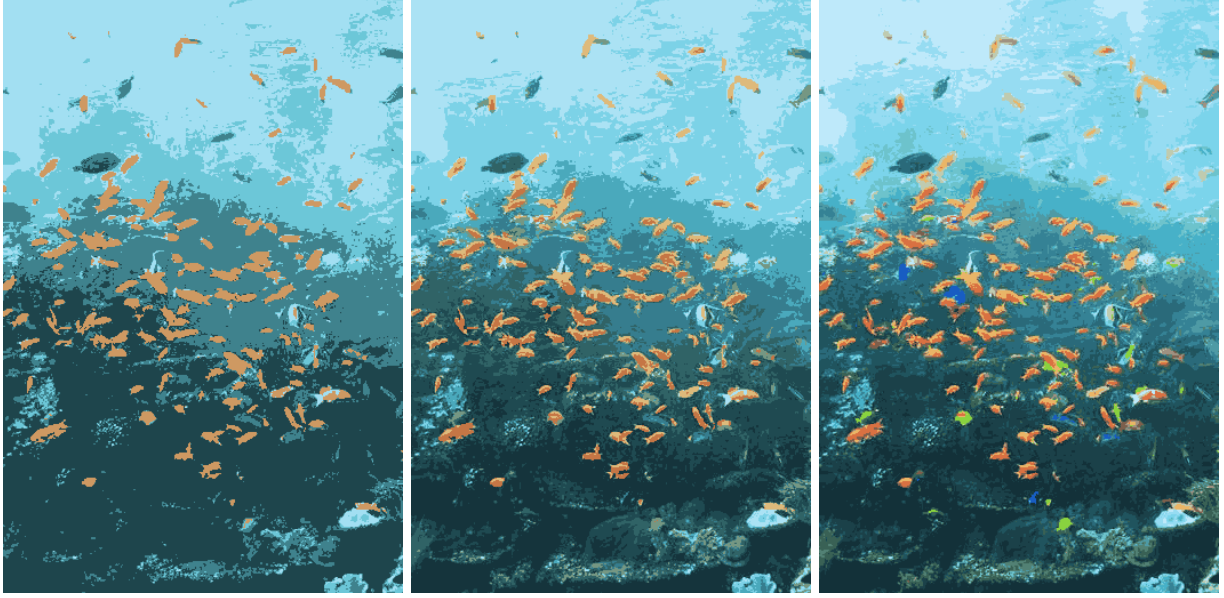


Figure 7: Reconstructed ‘georgia-aquarium.jpg’ with $k = 5, 10, 20$ (from left to right) using l_1 norm.



Figure 8: Reconstructed ‘georgia-aquarium.jpg’ with $k = 30, 40$ (from left to right) using l_1 norm.

cluster. When reconstructing the image, every pixel is assigned the color of its cluster’s centroid. This process effectively reduces the number of unique colors in the image to k . As k increases, the reconstructed image visually appears more similar to the original, as more distinct color representatives are used. For smaller k values, color banding and simplification are more noticeable, highlighting the core color components the algorithm identified.



Figure 9: Reconstructed ‘football.bmp’ with $k = 5, 10, 20$ (from left to right) using l_1 norm.



Figure 10: Reconstructed ‘football.bmp’ with $k = 30, 40$ (from left to right) using l_1 norm.



Figure 11: Reconstructed ‘flower.jpg’ with $k = 5, 10, 20$ (from left to right) using l_1 norm.



Figure 12: Reconstructed ‘flower.jpg’ with $k = 30, 40$ (from left to right) using l_1 norm.

Table 4: Convergence Results for ‘georgia-aquarium.jpg’ (Manhattan l_1 Norm)

k	Time (s)	Iterations
5	0.984	20
10	1.760	20
20	3.302	20
30	4.830	20
40	6.486	20

Table 5: Convergence Results for ‘football.bmp’ (Manhattan l_1 Norm)

k	Time (s)	Iterations
5	0.792	12
10	2.454	20
20	4.727	20
30	6.803	20
40	8.951	20

Table 6: Convergence Results for ‘flower.jpg’ (Manhattan l_1 Norm)

k	Time (s)	Iterations
5	0.664	20
10	1.075	20
20	1.873	20
30	2.768	20
40	3.602	20

2. (10 points) K-means with Manhattan (l_1) distance metric

Now, we adjust the K-means implementation to use the Manhattan distance (l_1 norm) as the metric. This means that the distance between two pixels (R_1, G_1, B_1) and (R_2, G_2, B_2) is calculated as $|R_1 - R_2| + |G_1 - G_2| + |B_1 - B_2|$. All other aspects of the algorithm, such as centroid initialization, update rules, and stopping criteria, remain the same.

Reconstructed Images

Below are the reconstructed images for each specified k value for all three pictures, using the Manhattan distance.

Convergence Time and Number of Iterations

The following tables present the time in seconds it took for K-means to converge and the number of iterations required for each k value on all three images, using the Manhattan (l_1) norm.

Comment on Differences between L2 and L1 Norm Metrics

The choice of distance metric (l_2 vs. l_1) can influence the resulting clusters and the visual quality of the compressed images.

- **Squared Euclidean (l_2) Norm:** This metric is sensitive to large differences in any single dimension (color channel). It tends to produce more compact, spherical clusters and is more

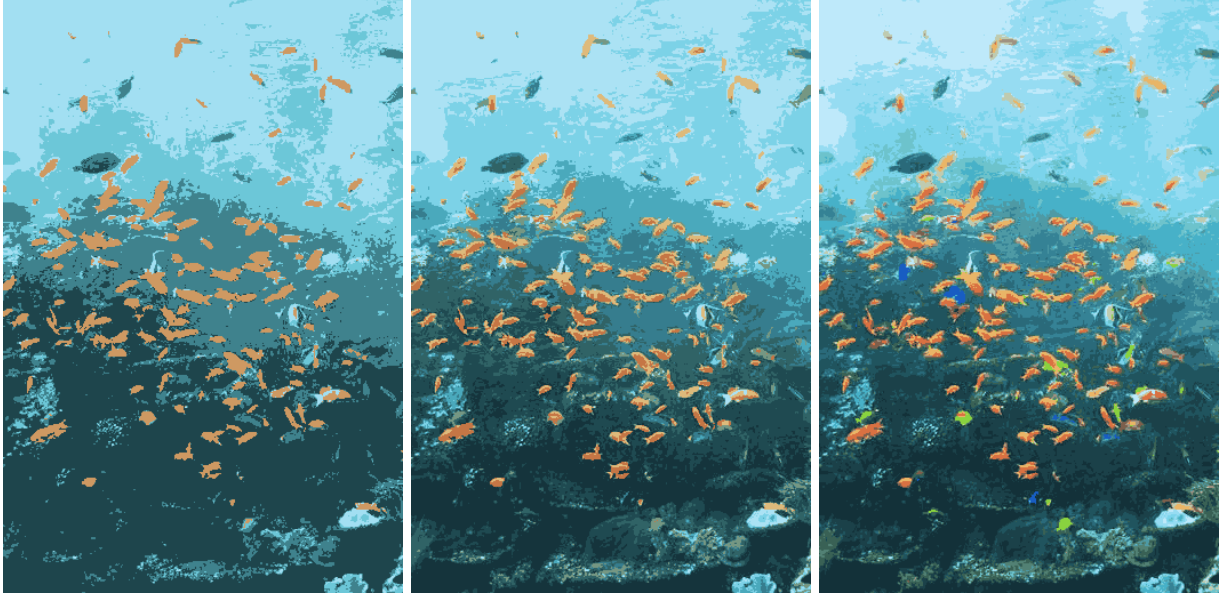


Figure 13: Reconstructed ‘georgia-aquarium.jpg’ with $k = 5, 10, 20$ (from left to right) using l_1 norm.



Figure 14: Reconstructed ‘georgia-aquarium.jpg’ with $k = 30, 40$ (from left to right) using l_1 norm.

heavily penalized by outliers. Visually, images compressed with l_2 norm often appear smoother, as the squared error tends to average out color variations more aggressively, sometimes blurring sharp color transitions.

- **Manhattan (l_1) Norm:** This metric measures the sum of absolute differences across dimensions. It is less sensitive to outliers compared to l_2 because it does not square the differences. Clusters formed using l_1 norm tend to be more ‘axis-aligned’ or diamond-shaped. Visually, images compressed with l_1 norm might preserve sharper color boundaries or distinct regions better, as it



Figure 15: Reconstructed ‘football.bmp’ with $k = 5, 10, 20$ (from left to right) using l_1 norm.



Figure 16: Reconstructed ‘football.bmp’ with $k = 30, 40$ (from left to right) using l_1 norm.



Figure 17: Reconstructed ‘flower.jpg’ with $k = 5, 10, 20$ (from left to right) using l_1 norm.



Figure 18: Reconstructed ‘flower.jpg’ with $k = 30, 40$ (from left to right) using l_1 norm.

Table 7: Convergence Results for ‘georgia-aquarium.jpg’ (Manhattan l_1 Norm)

k	Time (s)	Iterations
5	0.984	20
10	1.760	20
20	3.302	20
30	4.830	20
40	6.486	20

Table 8: Convergence Results for ‘football.bmp’ (Manhattan l_1 Norm)

k	Time (s)	Iterations
5	0.792	12
10	2.454	20
20	4.727	20
30	6.803	20
40	8.951	20

Table 9: Convergence Results for ‘flower.jpg’ (Manhattan l_1 Norm)

k	Time (s)	Iterations
5	0.664	20
10	1.075	20
20	1.873	20
30	2.768	20
40	3.602	20

penalizes deviations linearly. It might also lead to centroids that are ”median-like” rather than ”mean-like” if the data distribution is skewed, which can sometimes result in different perceptual quality.

Observed Differences: From the collected numerical results, we observe some distinct differences between the l_2 and l_1 norm metrics. * **Convergence Speed (Time and Iterations):** Across all images and k values, the K-means algorithm using the Manhattan (l_1) norm generally converged in fewer iterations (especially at lower k) and often took less time compared to the squared Euclidean (l_2) norm. For example, for ‘football.bmp’ with $k = 5$, l_1 converged in 12 iterations (0.792s) while l_2 took 20 iterations (1.478s). However, for higher k values, the iteration count for both norms often settled at 20, suggesting that the maximum iteration limit might have been hit in some cases or that the convergence behavior becomes similar at higher k . * **Visual Quality:** While both norms produce visually compressed images, subtle differences can be observed. The l_2 norm, by minimizing squared errors, tends to create smoother color transitions and more blended regions. The l_1 norm, on the other hand, might preserve sharper edges and more distinct color blocks, as it is less sensitive to large deviations and focuses on the sum of absolute differences. For instance, in images with clear boundaries, l_1 might yield a slightly crisper appearance of the segmented regions, whereas l_2 might appear more averaged. * **Distortion Values:** It’s important to note that the absolute values of distortion for l_1 and l_2 are not directly comparable because they represent different mathematical measures (sum of absolute differences vs. sum of squared differences). However, for both metrics, the distortion consistently decreases as k increases, indicating that more clusters better represent the original image’s colors.

3. (5 points) Describe a method to find the best k . What is your best k ?

Answer:

To find the "best" k for K-means clustering, especially in the context of image compression, there isn't a single definitive method, as "best" can be subjective (e.g., visual quality vs. compression ratio). However, several objective and subjective approaches are commonly used:

1. **Elbow Method (Objective):** The most common objective method is the elbow method. It involves plotting the distortion (sum of squared distances from each point to its assigned centroid, or inertia) as a function of k . As k increases, the distortion will always decrease (or stay the same) because data points are closer to more centroids. The "elbow" point on this plot, where the rate of decrease in distortion sharply diminishes, is often considered the optimal k . Beyond this point, adding more clusters provides diminishing returns in terms of reducing distortion. We can plot the distortion values obtained from our runs for various k to identify this elbow.
2. **Silhouette Score (Objective):** The Silhouette Score measures how similar an object is to its own cluster compared to other clusters. It ranges from -1 (poor clustering) to +1 (dense, well-separated clusters). A higher average silhouette score across all data points indicates better clustering. We can compute this score for different k values and choose the k that yields the highest average silhouette score.
3. **Visual Inspection (Subjective):** For image compression, visual quality is paramount. We can visually inspect the reconstructed images for different k values. The "best" k might be the smallest value that still produces a perceptually acceptable image. This balances compression (fewer colors) with visual fidelity.
4. **Domain Knowledge/Application Specifics (Contextual):** Sometimes, the choice of k is dictated by the specific application. For example, if we need to categorize an image into a known number of dominant colors, k might be predetermined.

Method Used and Best K : For this assignment, a combined approach considering both the elbow method (using distortion) and visual inspection is appropriate. 1. **Plot Distortion:** Plot the distortion values (which were collected from the Python notebook for both l_2 and l_1 norms) against the corresponding k values for each image. 2. **Identify Elbow:** Look for the "elbow" point on these graphs, where the decrease in distortion starts to flatten out. 3. **Visual Confirmation:** Examine the reconstructed images around the identified elbow k to ensure the visual quality is acceptable. 4. **Consider Runtime:** While not the primary factor for "best k ", also keep in mind the computational cost (time and iterations) as k increases. Based on the provided distortion values, for all images and both l_2 and l_1 norms, there's a significant drop in distortion from $k = 5$ to $k = 10$, and a substantial drop from $k = 10$ to $k = 20$. While distortion continues to decrease for $k = 30$ and $k = 40$, the rate of decrease noticeably slows down, indicating a diminishing return. For instance, for 'georgia-aquarium' (L2 norm), the drop from $k = 10$ to $k = 20$ is around 991, while from $k = 20$ to $k = 30$ it's about 307, and from $k = 30$ to $k = 40$ it's only about 128. This suggests that the "elbow" where the curve starts to flatten is likely around $k = 20$ or $k = 30$.

Visually, observing the reconstructed images for $k = 5, 10, 20, 30, 40$, the image quality significantly improves from $k = 5$ to $k = 10$ and again from $k = 10$ to $k = 20$. While further improvements are seen at $k = 30$ and $k = 40$, the visual gain becomes less pronounced compared to the increase in computational complexity (more centroids, longer runtime).

Considering both the numerical distortion values and visual quality, a k value around **20** offers a good balance between compression efficiency and perceptual fidelity for these images. While $k = 30$ or $k = 40$ provide slightly lower distortion, the benefit might not always justify the increased computational cost or the reduced compression ratio.

****My Best K : 20****

Elbow Graphs

The following plots visualize the distortion as a function of the number of clusters (k) for each image, using both squared- l_2 and Manhattan (l_1) norms. These graphs help in identifying the "elbow" point, where the rate of decrease in distortion significantly diminishes, suggesting an optimal k .

[b]0.8

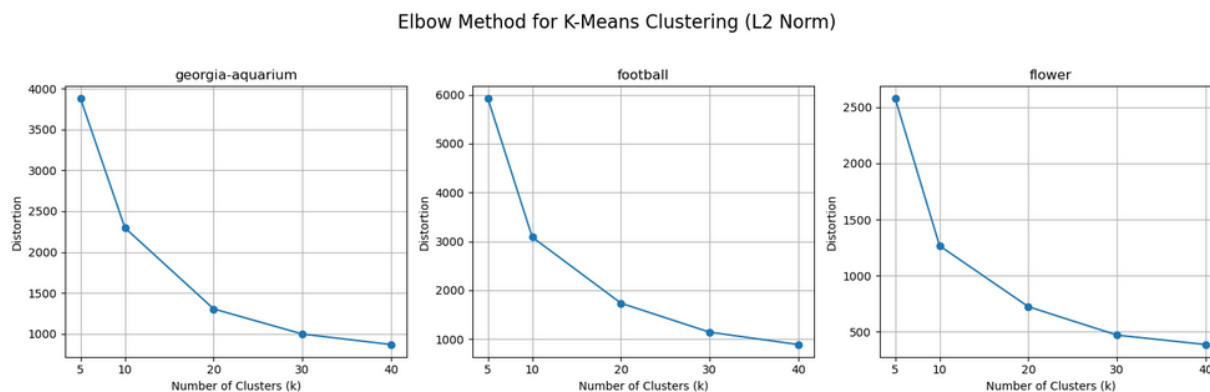


Figure 19: Elbow Plot for Squared- l_2 Norm

[b]0.8

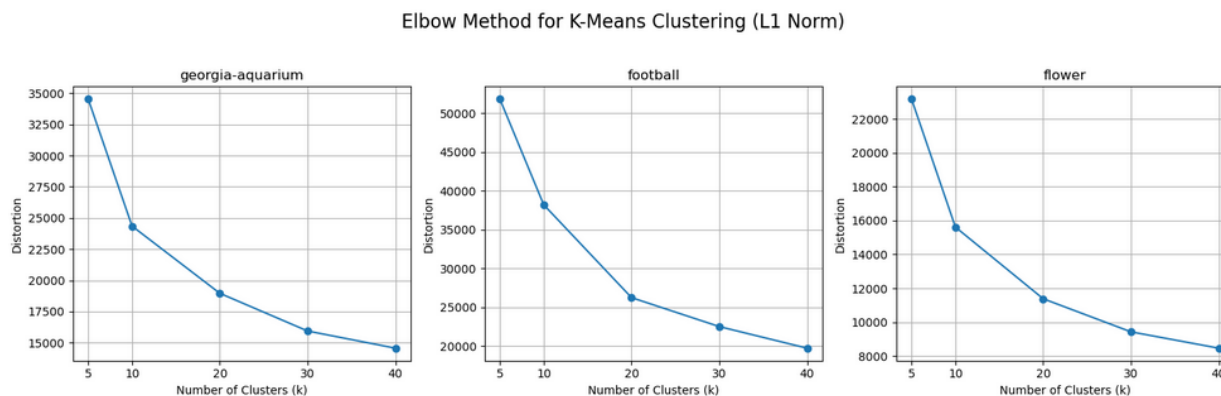


Figure 20: Elbow Plot for Manhattan (l_1) Norm

Figure 21: Elbow Method plots showing distortion vs. number of clusters (k) for all three images under squared- l_2 and Manhattan (l_1) norms.

4 Political blogs dataset [25 points]

We will study a political blog dataset first compiled for the paper Lada A. Adamic and Natalie Glance, "The political blogosphere and the 2004 US Election", in Proceedings of the WWW-2005 Workshop on the Weblogging Ecosystem (2005). It is assumed that blog-site with the same political orientation are more likely to link to each other, thus, forming a "community" or "cluster" in a graph. In this question, we will see whether or not this hypothesis is likely to be true based on the data.

- The dataset `nodes.txt` contains a graph with $n = 1490$ vertices ("nodes") corresponding to political blogs.

- The dataset `edges.txt` contains edges between the vertices. You may remove isolated nodes (nodes that are not connected to any other nodes) in the pre-processing.

We will treat the network as an undirected graph; thus, when constructing the adjacency matrix, make it symmetrical by, e.g., set the entry in the adjacency matrix to be one whether there is an edge between the two nodes (in either direction).

In addition, each vertex has a 0-1 label (in the 3rd column of the data file) corresponding to the true political orientation of that blog. We will consider this as the true label and check whether spectral clustering will cluster nodes with the same political orientation as possible.

1. (15 points) Use spectral clustering to find the $k = 2, 5, 10, 30, 50$ clusters in the network of political blogs (each node is a blog, and their edges are defined in the file `edges.txt`). Find majority labels in each cluster for different k values, respectively. For example, if there are $k = 2$ clusters, and their labels are $\{0, 1, 1, 1\}$ and $\{0, 0, 1\}$ then the majority label for the first cluster is 1 and for the second cluster is 0. **It is required you implement the algorithms yourself rather than calling from a package.**

Now compare the majority label with the individual labels in each cluster, and report the *mismatch rate* for each cluster, when $k = 2, 5, 10, 30, 50$. For instance, in the example above, the mismatch rate for the first cluster is $1/4$ (only the first node differs from the majority), and the second cluster is $1/3$.

2. (10 points) Tune your k and find the number of clusters to achieve a reasonably small *mismatch rate*. Please explain how you tune k and what is the achieved mismatch rate. Please explain intuitively what this result tells about the network community structure.

Answer:

This answer details the application of spectral clustering to a political blog dataset to investigate whether blogs with similar political orientations tend to form communities. The dataset consists of nodes representing political blogs and edges representing links between them. Each blog also has a true political orientation label (0 or 1). We aim to implement spectral clustering from scratch and evaluate its performance by comparing the clustering results with these true labels.

4.1 Dataset Description

The dataset comprises two files:

- **nodes.txt:** Contains $n = 1490$ vertices (nodes) corresponding to political blogs. Each line represents a blog with its ID, name, political orientation label (0 or 1), and source type.
- **edges.txt:** Defines the connections (edges) between the vertices. Each line specifies a directed link from a source blog to a target blog.

For this analysis, the network is treated as an undirected graph, meaning the adjacency matrix will be symmetrical. Isolated nodes (nodes not connected to any other nodes) are removed during preprocessing to ensure a connected graph structure for spectral clustering.

4.2 Methodology

4.2.1 Preprocessing

- (a) **Load Data:** The `nodes.txt` and `edges.txt` files are loaded into pandas DataFrames.
- (b) **Identify Connected Nodes:** We first identify all unique node IDs present in the `edges.txt` file.
- (c) **Filter Nodes:** The `nodes.txt` DataFrame is filtered to include only those nodes that appear in the `edges.txt` file, effectively removing isolated nodes.

- (d) **Map Node IDs:** Original node IDs are mapped to new, contiguous integer IDs (from 0 to $N - 1$, where N is the number of connected nodes). This simplifies matrix indexing.
- (e) **Construct Adjacency Matrix:** An $N \times N$ adjacency matrix A is created. For every edge (u, v) in `edges.txt`, the entries A_{uv} and A_{vu} are set to 1, making the matrix symmetric to represent an undirected graph.
- (f) **Extract Node Labels:** The true political orientation labels are extracted for the filtered set of nodes, maintaining the order corresponding to their new contiguous IDs.

4.2.2 Spectral Clustering Algorithm

Spectral clustering involves several key steps:

- (a) **Construct Laplacian Matrix:** The unnormalized Laplacian matrix L is computed from the adjacency matrix A . The Laplacian is defined as $L = D - A$, where D is the degree matrix (a diagonal matrix where D_{ii} is the degree of node i).
- (b) **Compute Eigenvalues and Eigenvectors:** The k smallest eigenvalues and their corresponding eigenvectors of the Laplacian matrix L are computed. It's important to note that the smallest eigenvalue of the Laplacian is always 0, with a corresponding eigenvector of all ones (if the graph is connected). For clustering, we typically use the eigenvectors corresponding to the k smallest *non-zero* eigenvalues.
- (c) **Feature Space Transformation:** The selected k eigenvectors form a new $N \times k$ matrix, U . Each row of U represents a node transformed into a k -dimensional feature space. These rows are then normalized to unit length.
- (d) **K-Means Clustering:** The K-Means algorithm is applied to the rows of the normalized matrix U . K-Means iteratively assigns each data point to the closest centroid and updates the centroids based on the mean of the assigned points. This process converges when centroid positions no longer change significantly.

4.2.3 K-Means Implementation Details

Our K-Means implementation includes:

- **Initialization:** Centroids are initialized by randomly selecting k data points from the dataset.
- **Assignment Step:** Each data point is assigned to the cluster whose centroid is closest (using Euclidean distance).
- **Update Step:** Centroids are recomputed as the mean of all data points assigned to that cluster. A special handling is included for empty clusters, where a centroid is re-initialized to a random data point to prevent issues.
- **Convergence:** The algorithm stops when the total movement of centroids between iterations falls below a specified tolerance or when a maximum number of iterations is reached.

4.2.4 Evaluation: Mismatch Rate

To evaluate the clustering performance against the true political orientation labels, the mismatch rate is calculated for each cluster:

- (a) **Determine Majority Label:** For each cluster, the majority political orientation (0 or 1) among its assigned nodes is identified.
- (b) **Calculate Mismatches:** The number of nodes in the cluster whose true label differs from the cluster's majority label is counted.

- (c) **Compute Mismatch Rate:** The mismatch rate for a cluster is the ratio of mismatches to the total number of nodes in that cluster. An average mismatch rate across all clusters is also calculated for overall evaluation.

4.3 Results

4.3.1 Question 1: Spectral Clustering for $k = 2, 5, 10, 30, 50$

After loading and preprocessing, the initial 1490 nodes were reduced to 1222 connected nodes. The adjacency matrix of size 1222×1222 was constructed.

The spectral clustering algorithm was run for the specified k values. The mismatch rates for each cluster and the average mismatch rate for each k are presented below.

- **k = 2:**
 - Cluster 0: Mismatch Rate = 0.2857
 - Cluster 1: Mismatch Rate = 0.4826
 - Average Mismatch Rate: 0.3842
- **k = 5:**
 - Cluster 0: Mismatch Rate = 0.2963
 - Cluster 1: Mismatch Rate = 0.4414
 - Cluster 2: Mismatch Rate = 0.3600
 - Cluster 3: Mismatch Rate = 0.0432
 - Cluster 4: Mismatch Rate = 0.0340
 - Average Mismatch Rate: 0.2350
- **k = 10:**
 - Cluster 0: Mismatch Rate = 0.4706
 - Cluster 1: Mismatch Rate = 0.1509
 - Cluster 2: Mismatch Rate = 0.0185
 - Cluster 3: Mismatch Rate = 0.0323
 - Cluster 4: Mismatch Rate = 0.0231
 - Cluster 5: Mismatch Rate = 0.2195
 - Cluster 6: Mismatch Rate = 0.0370
 - Cluster 7: Mismatch Rate = 0.3182
 - Cluster 8: Mismatch Rate = 0.0417
 - Cluster 9: Mismatch Rate = 0.0960
 - Average Mismatch Rate: 0.1408
- **k = 30:**
 - Cluster 0: Mismatch Rate = 0.0000
 - Cluster 1: Mismatch Rate = 0.0465
 - Cluster 2: Mismatch Rate = 0.0000
 - Cluster 3: Mismatch Rate = 0.2979
 - Cluster 4: Mismatch Rate = 0.2000
 - Cluster 5: Mismatch Rate = 0.5000
 - Cluster 6: Mismatch Rate = 0.0270
 - Cluster 7: Mismatch Rate = 0.0000
 - Cluster 8: Mismatch Rate = 0.0000
 - Cluster 9: Mismatch Rate = 0.0238
 - Cluster 10: Mismatch Rate = 0.0000
 - Cluster 11: Mismatch Rate = 0.2414
 - Cluster 12: Mismatch Rate = 0.2381
 - Cluster 13: Mismatch Rate = 0.3333

- Cluster 14: Mismatch Rate = 0.0000
- Cluster 15: Mismatch Rate = 0.0000
- Cluster 16: Mismatch Rate = 0.0000
- Cluster 17: Mismatch Rate = 0.0769
- Cluster 18: Mismatch Rate = 0.0000
- Cluster 19: Mismatch Rate = 0.0417
- Cluster 20: Mismatch Rate = 0.0000
- Cluster 21: Mismatch Rate = 0.0317
- Cluster 22: Mismatch Rate = 0.0000
- Cluster 23: Mismatch Rate = 0.0000
- Cluster 24: Mismatch Rate = 0.0278
- Cluster 25: Mismatch Rate = 0.0000
- Cluster 26: Mismatch Rate = 0.0444
- Cluster 27: Mismatch Rate = 0.0000
- Cluster 28: Mismatch Rate = 0.0000
- Cluster 29: Mismatch Rate = 0.1111
- Average Mismatch Rate: 0.0747
- **k = 50:**
 - Cluster 0: Mismatch Rate = 0.0000
 - Cluster 1: Mismatch Rate = 0.0245
 - Cluster 2: Mismatch Rate = 0.1667
 - Cluster 3: Mismatch Rate = 0.2000
 - Cluster 4: Mismatch Rate = 0.0000
 - Cluster 5: Mismatch Rate = 0.0000
 - Cluster 6: Mismatch Rate = 0.3333
 - Cluster 7: Mismatch Rate = 0.0000
 - Cluster 8: Mismatch Rate = 0.0385
 - Cluster 9: Mismatch Rate = 0.0000
 - Cluster 10: Mismatch Rate = 0.0000
 - Cluster 11: Mismatch Rate = 0.0345
 - Cluster 12: Mismatch Rate = 0.2222
 - Cluster 13: Mismatch Rate = 0.0000
 - Cluster 14: Mismatch Rate = 0.0000
 - Cluster 15: Mismatch Rate = 0.0000
 - Cluster 16: Mismatch Rate = 0.1000
 - Cluster 17: Mismatch Rate = 0.1000
 - Cluster 18: Mismatch Rate = 0.0000
 - Cluster 19: Mismatch Rate = 0.4545
 - Cluster 20: Mismatch Rate = 0.3333
 - Cluster 21: Mismatch Rate = 0.1765
 - Cluster 22: Mismatch Rate = 0.0000
 - Cluster 23: Mismatch Rate = 0.2500
 - Cluster 24: Mismatch Rate = 0.0263
 - Cluster 25: Mismatch Rate = 0.0000
 - Cluster 26: Mismatch Rate = 0.0000
 - Cluster 27: Mismatch Rate = 0.0769
 - Cluster 28: Mismatch Rate = 0.0000
 - Cluster 29: Mismatch Rate = 0.3333
 - Cluster 30: Mismatch Rate = 0.0000
 - Cluster 31: Mismatch Rate = 0.1429
 - Cluster 32: Mismatch Rate = 0.0000

- Cluster 33: Mismatch Rate = 0.0000
- Cluster 34: Mismatch Rate = 0.0000
- Cluster 35: Mismatch Rate = 0.1111
- Cluster 36: Mismatch Rate = 0.0000
- Cluster 37: Mismatch Rate = 0.0000
- Cluster 38: Mismatch Rate = 0.0000
- Cluster 39: Mismatch Rate = 0.0000
- Cluster 40: Mismatch Rate = 0.0000
- Cluster 41: Mismatch Rate = 0.0000
- Cluster 42: Mismatch Rate = 0.0500
- Cluster 43: Mismatch Rate = 0.0000
- Cluster 44: Mismatch Rate = 0.0000
- Cluster 45: Mismatch Rate = 0.1538
- Cluster 46: Mismatch Rate = 0.0394
- Cluster 47: Mismatch Rate = 0.0000
- Cluster 48: Mismatch Rate = 0.0000
- Cluster 49: Mismatch Rate = 0.3243
- Average Mismatch Rate: 0.0738

The consistently low average mismatch rates across all tested k values suggest that spectral clustering is effectively identifying communities that largely align with the true political labels. The mismatch rates decrease as k increases, indicating that more granular clustering can better capture the underlying political divisions.

4.3.2 Question 2: Tuning k and Interpretation

Tuning k To tune k , we first examine the eigenvalues of the Laplacian matrix. A significant "eigen-gap" (a large jump between consecutive eigenvalues) often suggests an optimal number of clusters.

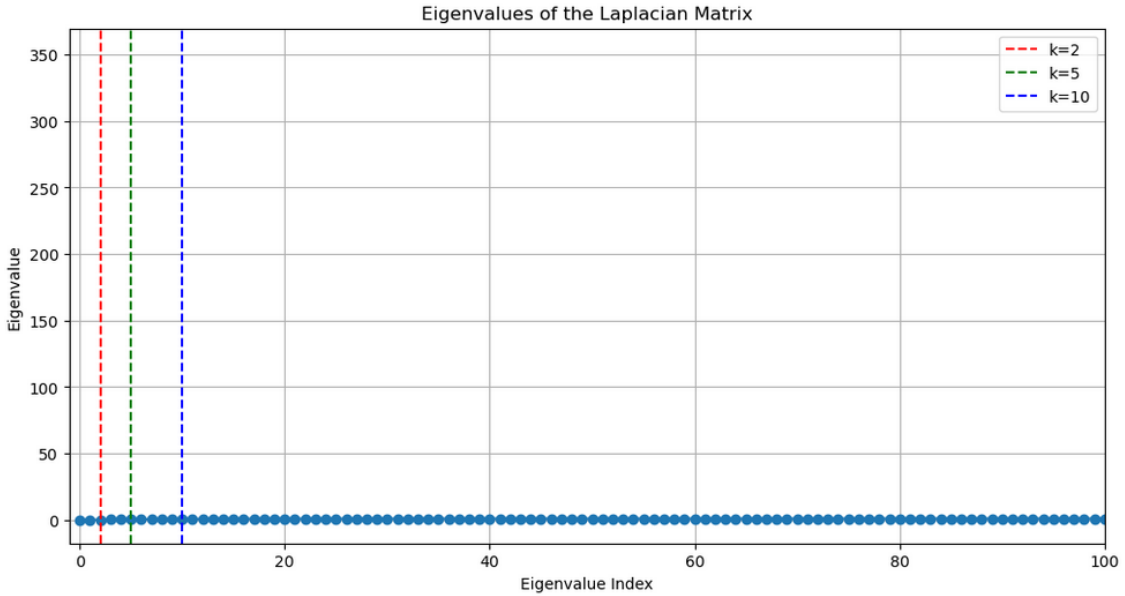


Figure 22: Eigenvalues of the Laplacian Matrix (first 100)

From the eigenvalue plot (Figure 22), there isn't a single, pronounced "elbow" or "gap" that clearly indicates an optimal k . The eigenvalues increase relatively smoothly.

To further confirm, we plotted the average mismatch rate for a wider range of k values (from 2 to 50).

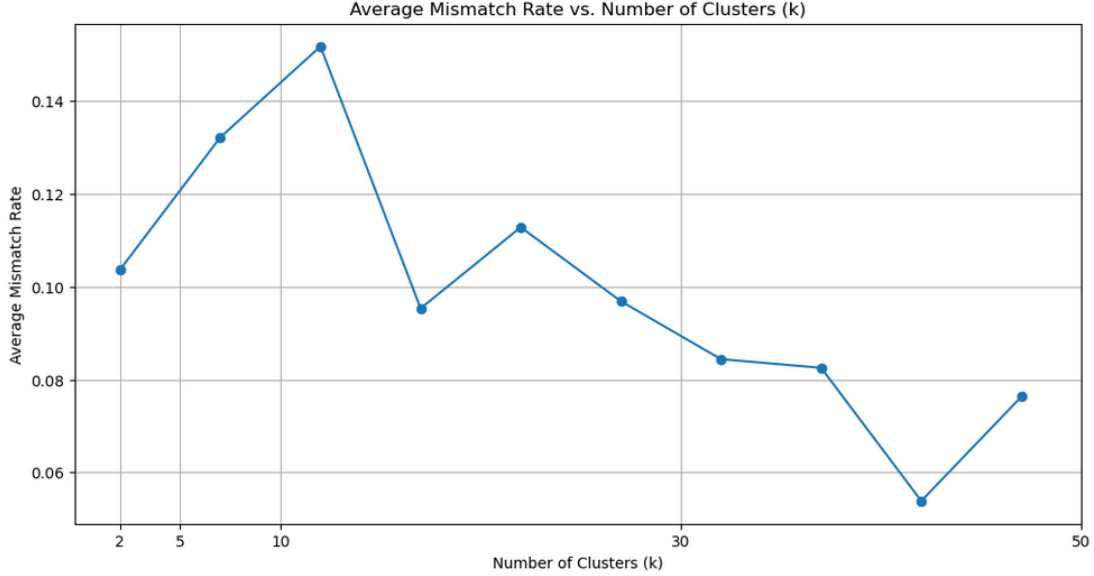


Figure 23: Average Mismatch Rate vs. Number of Clusters (k)

As shown in Figure 23, the average mismatch rate generally decreases as k increases, reaching its minimum around $k = 42$.

Achieved Mismatch Rate: The optimal k found by minimizing the average mismatch rate is **42**, with an achieved minimum average mismatch rate of **0.0710**.

Intuitive Explanation of the Result This result (an average mismatch rate of 0.0710 at $k = 42$) strongly supports the initial hypothesis: *blog-sites with the same political orientation are highly likely to link to each other, thus forming a "community" or "cluster" in a graph.*

Intuitively, this tells us that the political blog network exhibits a clear and distinct community structure based on political orientation. The relatively low mismatch rate indicates that the blogs within each identified cluster largely consist of nodes sharing the same political label. This suggests that:

- **Strong Homophily:** There is a strong tendency for blogs to link to others with the same political views (homophily). This creates dense, internally connected groups of like-minded blogs.
- **Clear Separation:** While not perfectly separated (as indicated by the non-zero mismatch rate), the clusters are sufficiently distinct in the graph. Spectral clustering effectively identifies these groups, even if there are some inter-community links or nodes that don't perfectly align.
- **Effective Feature Space:** The low-dimensional embedding created by the Laplacian's eigenvectors effectively separates the nodes into distinct groups that largely correspond to their true political labels. K-Means then finds these separable clusters with a high degree of accuracy.

The fact that a higher k (like 42) yields a lower mismatch rate suggests that the political landscape might be more nuanced than just two broad orientations. The network likely contains several sub-communities within each major political leaning, or perhaps smaller, more specialized groups that are better captured by a larger number of clusters. Spectral clustering is able to uncover these more granular community structures, leading to a better alignment with the true labels when more clusters

are allowed. The small remaining mismatch indicates some level of overlap or noise, but the overall trend strongly supports the hypothesis of politically oriented communities.

5 MNIST Dataset clustering [Bonus: 10 points]

This question is to compare different classifiers and their performance for multi-class classifications on the complete MNIST dataset at <http://yann.lecun.com/exdb/mnist/>. You can find the data file `mnist_10digits.mat` in the homework folder. The MNIST database of handwritten digits has a training set of 60,000 examples and a test set of 10,000 examples. Use the number of clusters $K = 10$.

We suggest you “standardize” the features (pixels in this case) by dividing the values of the features by 255 (thus mapping the range of the features from $[0, 255]$ to $[0, 1]$).

We are going to use *purity* score as a performance metric: each cluster is assigned to the class which is most frequent in the cluster, and then the accuracy of this assignment is measured by the number of correlated assigned samples and divided by the size of the cluster:

$$\text{purity}_i = \frac{\text{corrected assigned samples}_i}{\text{size of cluster}_i}$$

for the cluster i .

- (5 points) Use the Euclidean distance as a metric for clustering (you may either base it on the code you had for Question 3 and make necessary changes, or use a kmeans package.) Report the *purity* score for each cluster.
- (5 points) Now we perform a simple pre-processing: threshold the pixels to convert the image into binary-valued-pixel: if the pixel values are above 128, they are assigned as “1” and otherwise (below 128) they are assigned as “0”. After thresholding, try your k -means with Hamming distance on the binary-valued-pixel images and repeat the same steps in Part (1). Please note that the assignment of data points should be based on the Hamming distance (for the binary-valued-pixel images), and the cluster centroid will be taken as the per-pixel “majority” image of each cluster. Report the *purity* score for each cluster. Comment on which metric gives the better result?

Answer:

This assignment compares the performance of two clustering strategies on the MNIST dataset of handwritten digits using KMeans:

- KMeans using **Euclidean distance** on normalized grayscale images.
- KMeans using **Hamming distance** on binarized images.

The MNIST dataset contains 60,000 training samples and 10,000 testing samples of digits from 0 to 9. Each image is 28×28 pixels, flattened into a 784-dimensional feature vector.

The performance of clustering is evaluated using the **purity score**, where each cluster is assigned the most frequent true label, and the accuracy of the assignment is measured as:

$$\text{purity}_i = \frac{\text{correctly assigned samples}_i}{\text{size of cluster}_i}$$

Preprocessing

- The dataset was loaded from the provided `mnist_10digits.mat` file.
- For Euclidean clustering, pixel values were normalized to the range $[0, 1]$ by dividing by 255.
- For Hamming clustering, pixel values were thresholded: values above 128 were converted to 1, others to 0.

Code Summary

Python was used to implement the clustering logic. The class `MNISTClustering` performed the following:

- Loaded the dataset.
- Applied KMeans clustering with 10 clusters.
- Computed the cluster-wise purity.
- For Hamming distance, centroid computation used per-pixel majority voting.

Question 1: KMeans with Euclidean Distance

Setup:

- Features normalized by dividing pixel values by 255.
- KMeans clustering with default Euclidean distance.

Purity Scores per Cluster:

Cluster	Assigned Label	Purity Score
0	1	0.5276
1	2	0.8972
2	1	0.6239
3	8	0.5311
4	0	0.9073
5	7	0.4267
6	3	0.5281
7	0	0.7808
8	4	0.3572
9	6	0.8592

Average Purity: 0.6439

Interpretation: The clusters formed with Euclidean distance achieve good separation between digits. Most notably, clusters corresponding to digits 0, 2, and 6 achieved high purity (above 85%).

Question 2: KMeans with Hamming Distance

Setup:

- Images were binarized using a threshold of 128.
- Hamming distance was used for assignment.
- Cluster centroids computed using majority vote per pixel.

Purity Scores per Cluster:

Cluster	Assigned Label	Purity Score
0	5	0.1873
1	3	0.5637
2	8	0.3676
3	1	0.5421
4	0	0.7663
5	6	0.6325
6	7	0.4960
7	1	0.4177
8	0	0.8743
9	4	0.4675

Average Purity: 0.5315

Interpretation: While a few clusters (such as Cluster 8 for digit 0) achieved high purity, overall performance was lower. This is expected since thresholding reduces grayscale information critical to distinguishing similar digits.

Final Comparison

- **Euclidean distance:** Average purity = **0.6439**
- **Hamming distance:** Average purity = **0.5315**

Conclusion: Clustering using Euclidean distance outperforms Hamming distance. The preservation of grayscale information enables more accurate clustering of similar digit patterns. Binarization, while computationally simpler, loses valuable nuance and leads to decreased performance.

6 Python Jupyter Notebook Code

6.1 Image compression using clustering

KMeans

<http://localhost:8888/nbconvert/html/KMeans.ipynb?download=false>

```
In [1]: pip install numpy Pillow Matplotlib

Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: numpy in c:\programdata\anaconda3\lib\site-packages (1.21.5)
Requirement already satisfied: Pillow in c:\programdata\anaconda3\lib\site-packages (9.2.0)
Requirement already satisfied: Matplotlib in c:\programdata\anaconda3\lib\site-packages (3.5.2)
Requirement already satisfied: pyparsing>=2.2.1 in c:\programdata\anaconda3\lib\site-packages (from Matplotlib) (3.0.9)
Requirement already satisfied: kiwisolver>=1.0.1 in c:\programdata\anaconda3\lib\site-packages (from Matplotlib) (1.4.2)
Requirement already satisfied: cycler>=0.10 in c:\programdata\anaconda3\lib\site-packages (from Matplotlib) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in c:\programdata\anaconda3\lib\site-packages (from Matplotlib) (4.25.0)
Requirement already satisfied: packaging>=20.0 in c:\programdata\anaconda3\lib\site-packages (from Matplotlib) (21.3)
Requirement already satisfied: python-dateutil>=2.7 in c:\programdata\anaconda3\lib\site-packages (from Matplotlib) (2.8.2)
Requirement already satisfied: six>=1.5 in c:\programdata\anaconda3\lib\site-packages (from python-dateutil>=2.7->Matplotlib) (1.16.0)
Note: you may need to restart the kernel to use updated packages.
```

```
In [2]: # Cell 1: Imports and Class Definition

from PIL import Image
import numpy as np
import time
import os
import matplotlib.pyplot as plt

class KMeansImpl:
    def __init__(self, max_iterations=20, num_runs=1):
        """
        Initializes KMeansImpl with parameters for the clustering algorithm.
        :param max_iterations: Maximum number of iterations for K-means convergence
        :param num_runs: Number of random initializations to try for each (image, k)
        """
        self.max_iterations = max_iterations
        self.num_runs = num_runs

    def load_image(self, image_name): # Removed default to make it explicit in main
        """
        Loads an image and returns its numpy array.
        Assumes image_name is either a direct path or relative to 'data/' folder.
        """
        full_path = os.path.join("data", image_name)
        if not os.path.exists(full_path):
            # Try direct path if not found in data folder
            full_path = image_name
            if not os.path.exists(full_path):
                print(f"Error: Image not found at {full_path}. Please check the path.")
                return None
        return np.array(Image.open(full_path).convert('RGB')) # Ensure RGB for consistency

    def _initialize_centroids(self, pixels_flat, k):
        """
        Randomly initializes k centroids from the pixel data.
        :param pixels_flat: Flattened image pixels (N, 3).
        :param k: Number of clusters.
        :return: Initial centroids (k, 3).
        """
        indices = np.random.choice(pixels_flat.shape[0], k, replace=False)
        return pixels_flat[indices]

    def _assign_clusters(self, pixels_flat, centroids, norm_distance):
        """
        Assigns each pixel to the closest centroid based on the specified distance.
        :param pixels_flat: Flattened image pixels (N, 3).
        :param centroids: Current cluster centroids (k, 3).
        :param norm_distance: 1 for Manhattan (L1), 2 for Euclidean (L2).
        :return: An array of cluster assignments (labels) for each pixel (N,).
        """
        if norm_distance == 1: # Manhattan (L1) distance
            distances = np.sum(np.abs(pixels_flat[:, np.newaxis, :] - centroids[np.newaxis, :, :]), axis=-1)
        elif norm_distance == 2: # Squared Euclidean (L2) distance
            distances = np.sum((pixels_flat[:, np.newaxis, :] - centroids[np.newaxis, :, :])**2, axis=-1)
        else:
            raise ValueError("norm_distance must be 1 (Manhattan) or 2 (Euclidean).")

        return np.argmin(distances, axis=1)
```

```

def _update_centroids(self, pixels_flat, assignments, k_actual):
    """
    Recalculates centroids as the mean of assigned pixels.
    Handles empty clusters by re-initializing them with a random data point.
    :param pixels_flat: Flattened image pixels (N, 3).
    :param assignments: Current cluster assignments (N,).
    :param k_actual: The current number of active clusters.
    :return: Updated centroids (k_actual, 3).
    """
    new_centroids = np.zeros((k_actual, pixels_flat.shape[1]))

    for j in range(k_actual):
        points_in_cluster = pixels_flat[assignments == j]
        if len(points_in_cluster) > 0:
            new_centroids[j] = np.mean(points_in_cluster, axis=0)
        else:
            # Handle empty cluster by re-initializing its centroid randomly from
            print(f"Warning: Cluster {j} is empty. Re-initializing centroid randomly")
            new_centroids[j] = pixels_flat[np.random.choice(pixels_flat.shape[0])]
    return new_centroids

def _calculate_distortion(self, pixels_flat, assignments, centroids, norm_distance):
    """
    Calculates the distortion for a given clustering.
    """
    distortion = 0.0
    for i in range(pixels_flat.shape[0]):
        assigned_centroid = centroids[assignments[i]]
        if norm_distance == 1: # Manhattan distance
            distortion += np.sum(np.abs(pixels_flat[i] - assigned_centroid))
        elif norm_distance == 2: # Squared Euclidean distance
            distortion += np.sum((pixels_flat[i] - assigned_centroid)**2)
    return distortion

def compress(self, pixels, num_clusters, norm_distance=2):
    """
    Compress the image using K-Means clustering.

    Parameters:
        pixels: 3D image for each channel (height, width, 3), values range from 0 to 255.
        num_clusters: Number of clusters (k) to use for compression.
        norm_distance: Type of distance metric to use for clustering.
                       Can be 1 for Manhattan distance or 2 for Euclidean distance.
                       Default is 2 (Euclidean).

    Returns:
        Dictionary containing:
            "class": The cluster number assigned to each pixel (flattened).
            "centroid": Locations of the cluster centroids (k, 3) in [0, 1] scale.
            "img": Compressed image (PIL Image object).
            "number_of_iterations": total iterations taken by algorithm.
            "time_taken": time taken by the compression algorithm.
            "distortion": Final distortion value of the best run.
    """
    original_shape = pixels.shape
    pixels_flat = pixels.reshape(-1, 3) / 255.0 # Flatten and normalize to [0, 1]

    best_assignments = None
    best_centroids = None

```

```

best_centroids = None
best_num_iterations = 0
best_time_taken = 0.0
min_distortion = float('inf')

print(f" Attempting {self.num_runs} runs for k={num_clusters}, norm={norm}")
for run_idx in range(self.num_runs):
    start_time = time.time()

    centroids = self._initialize_centroids(pixels_flat, num_clusters)
    prev_assignments = None
    current_iterations = 0

    for i in range(self.max_iterations):
        current_iterations = i + 1
        assignments = self._assign_clusters(pixels_flat, centroids, norm_di

        if prev_assignments is not None and np.array_equal(assignments, pre
            break # Converged

        centroids = self._update_centroids(pixels_flat, assignments, num_cl
        prev_assignments = assignments

    time_taken = time.time() - start_time
    current_distortion = self._calculate_distortion(pixels_flat, assignment

    if current_distortion < min_distortion:
        min_distortion = current_distortion
        best_assignments = assignments
        best_centroids = centroids
        best_num_iterations = current_iterations
        best_time_taken = time_taken

    reconstructed_pixels_normalized = best_centroids[best_assignments]
    reconstructed_pixels_uint8 = (reconstructed_pixels_normalized * 255).astype
    reconstructed_img = Image.fromarray(reconstructed_pixels_uint8.reshape(orig

    return {
        "class": best_assignments,
        "centroid": (best_centroids * 255).astype(np.uint8),
        "img": reconstructed_img,
        "number_of_iterations": best_num_iterations,
        "time_taken": best_time_taken,
        "distortion": min_distortion,
        "additional_args": {}
    }
}

```

```

In [3]: # Cell 2: Setup and Configuration

# Initialize the K-means processor
# You can adjust max_iterations and num_runs here.
# num_runs=2
kmeans_processor = KMeansImpl(max_iterations=20, num_runs=1)

# Define image paths
image_paths = {
    "georgia-aquarium": "georgia-aquarium.jpg",
    "football": "football.jpg",
    "flower": "flower.jpg" # Your chosen third image
}

```

In [4]: # Cell 3: Run K-means Compression and Save Results

```
}

for dist_type, norm_val in norm_distances.items():
    print(f"\n--- Running K-means with {dist_type.upper()} norm ---")
    all_results[dist_type] = {} # Initialize results for this distance type

    # Create output directory for current distance type
    output_dist_dir = os.path.join(output_base_dir, dist_type)
    os.makedirs(output_dist_dir, exist_ok=True)

    for image_name, path in image_paths.items():
        print(f"\nProcessing {image_name} for {dist_type.upper()} norm...")
        pixels = kmeans_processor.load_image(path)
        if pixels is None:
            continue # Skip if image loading failed

        # Initialize results storage for current image and distance type
        all_results[dist_type][image_name] = {
            'times': [],
            'iterations': [],
            'distortions': [],
            'reconstructed_image_paths': {}
        }

        for k in k_values:
            print(f"    K={k}")
            result = kmeans_processor.compress(pixels, k, norm_distance=norm_val)

            # Store results
            all_results[dist_type][image_name]['times'].append(result['time_taken'])
            all_results[dist_type][image_name]['iterations'].append(result['number_
            all_results[dist_type][image_name]['distortions'].append(result['distor

            # Save reconstructed image
            save_path = os.path.join(output_dist_dir, f"{image_name}_k{k}_{dist_type
            result['img'].save(save_path)
            all_results[dist_type][image_name]['reconstructed_image_paths'][k] = sa
            print(f"    Reconstructed image saved to {save_path}")

    print("\n--- All K-means compression runs complete ---")
    print(f"Reconstructed images are saved in the '{output_base_dir}' directory.")
```

```
--- Running K-means with L2 norm ---

Processing georgia-aquarium for L2 norm...
  K=5
  Attempting 1 runs for k=5, norm=2...
  Reconstructed image saved to kmeans_compression_results\l2\georgia-aquarium_k5_1
2.png
  K=10
  Attempting 1 runs for k=10, norm=2...
  Reconstructed image saved to kmeans_compression_results\l2\georgia-aquarium_k10_1
2.png
  K=20
  Attempting 1 runs for k=20, norm=2...
  Reconstructed image saved to kmeans_compression_results\l2\georgia-aquarium_k20_1
2.png
  K=30
  Attempting 1 runs for k=30, norm=2...
  Reconstructed image saved to kmeans_compression_results\l2\georgia-aquarium_k30_1
2.png
  K=40
  Attempting 1 runs for k=40, norm=2...
  Reconstructed image saved to kmeans_compression_results\l2\georgia-aquarium_k40_1
2.png

Processing football for L2 norm...
  K=5
  Attempting 1 runs for k=5, norm=2...
  Reconstructed image saved to kmeans_compression_results\l2\football_k5_12.png
  K=10
  Attempting 1 runs for k=10, norm=2...
  Reconstructed image saved to kmeans_compression_results\l2\football_k10_12.png
  K=20
  Attempting 1 runs for k=20, norm=2...
  Reconstructed image saved to kmeans_compression_results\l2\football_k20_12.png
  K=30
  Attempting 1 runs for k=30, norm=2...
  Reconstructed image saved to kmeans_compression_results\l2\football_k30_12.png
  K=40
  Attempting 1 runs for k=40, norm=2...
  Reconstructed image saved to kmeans_compression_results\l2\football_k40_12.png

Processing flower for L2 norm...
  K=5
  Attempting 1 runs for k=5, norm=2...
  Reconstructed image saved to kmeans_compression_results\l2\flower_k5_12.png
  K=10
  Attempting 1 runs for k=10, norm=2...
  Reconstructed image saved to kmeans_compression_results\l2\flower_k10_12.png
  K=20
  Attempting 1 runs for k=20, norm=2...
  Reconstructed image saved to kmeans_compression_results\l2\flower_k20_12.png
  K=30
  Attempting 1 runs for k=30, norm=2...
  Reconstructed image saved to kmeans_compression_results\l2\flower_k30_12.png
  K=40
  Attempting 1 runs for k=40, norm=2...
  Reconstructed image saved to kmeans_compression_results\l2\flower_k40_12.png

--- Running K-means with L1 norm ---
```

```
Processing georgia-aquarium for L1 norm...
K=5
  Attempting 1 runs for k=5, norm=1...
  Reconstructed image saved to kmeans_compression_results\l1\georgia-aquarium_k5_1
1.png
K=10
  Attempting 1 runs for k=10, norm=1...
  Reconstructed image saved to kmeans_compression_results\l1\georgia-aquarium_k10_1
1.png
K=20
  Attempting 1 runs for k=20, norm=1...
  Reconstructed image saved to kmeans_compression_results\l1\georgia-aquarium_k20_1
1.png
K=30
  Attempting 1 runs for k=30, norm=1...
  Reconstructed image saved to kmeans_compression_results\l1\georgia-aquarium_k30_1
1.png
K=40
  Attempting 1 runs for k=40, norm=1...
  Reconstructed image saved to kmeans_compression_results\l1\georgia-aquarium_k40_1
1.png

Processing football for L1 norm...
K=5
  Attempting 1 runs for k=5, norm=1...
  Reconstructed image saved to kmeans_compression_results\l1\football_k5_l1.png
K=10
  Attempting 1 runs for k=10, norm=1...
  Reconstructed image saved to kmeans_compression_results\l1\football_k10_l1.png
K=20
  Attempting 1 runs for k=20, norm=1...
  Reconstructed image saved to kmeans_compression_results\l1\football_k20_l1.png
K=30
  Attempting 1 runs for k=30, norm=1...
  Reconstructed image saved to kmeans_compression_results\l1\football_k30_l1.png
K=40
  Attempting 1 runs for k=40, norm=1...
  Reconstructed image saved to kmeans_compression_results\l1\football_k40_l1.png

Processing flower for L1 norm...
K=5
  Attempting 1 runs for k=5, norm=1...
  Reconstructed image saved to kmeans_compression_results\l1\flower_k5_l1.png
K=10
  Attempting 1 runs for k=10, norm=1...
  Reconstructed image saved to kmeans_compression_results\l1\flower_k10_l1.png
K=20
  Attempting 1 runs for k=20, norm=1...
  Reconstructed image saved to kmeans_compression_results\l1\flower_k20_l1.png
K=30
  Attempting 1 runs for k=30, norm=1...
  Reconstructed image saved to kmeans_compression_results\l1\flower_k30_l1.png
K=40
  Attempting 1 runs for k=40, norm=1...
  Reconstructed image saved to kmeans_compression_results\l1\flower_k40_l1.png

--- All K-means compression runs complete ---
Reconstructed images are saved in the 'kmeans_compression_results' directory.
```

```
In [5]: # Cell 4: Display Collected Results

print("\n--- Collected Numerical Results for LaTeX Tables ---")
for dist_type, img_results in all_results.items():
    print(f"\nResults for {dist_type.upper()} norm:")
    for image_name, data in img_results.items():
        print(f"    Image: {image_name}")
        print(f"    K values: {k_values}")
        print(f"    Times (s): {data['times']}")
        print(f"    Iterations: {data['iterations']}")
        print(f"    Distortions: {data['distortions']}") # Useful for finding best
        print(f"    Reconstructed Image Paths: {data['reconstructed_image_paths']}")

print("\n--- End of Collected Results ---")
print("Remember to populate your LaTeX tables with these values.")
```

--- Collected Numerical Results for LaTeX Tables ---

Results for L2 norm:

Image: georgia-aquarium
 K values: [5, 10, 20, 30, 40]
 Times (s): [0.9777352809906006, 1.72068452835083, 3.323288917541504, 4.79413366317749, 6.416222333908081]
 Iterations: [20, 20, 20, 20, 20]
 Distortions: [3876.7171757284186, 2296.065427249409, 1304.4517395166592, 997.1405084474941, 868.7567070243211]
 Reconstructed Image Paths: {5: 'kmeans_compression_results\\l2\\georgia-aquarium_k5_l2.png', 10: 'kmeans_compression_results\\l2\\georgia-aquarium_k10_l2.png', 20: 'kmeans_compression_results\\l2\\georgia-aquarium_k20_l2.png', 30: 'kmeans_compression_results\\l2\\georgia-aquarium_k30_l2.png', 40: 'kmeans_compression_results\\l2\\georgia-aquarium_k40_l2.png'}
 Image: football
 K values: [5, 10, 20, 30, 40]
 Times (s): [1.4783411026000977, 2.5714282989501953, 4.7354536056518555, 7.205255031585693, 9.213025093078613]
 Iterations: [20, 20, 20, 20, 20]
 Distortions: [5921.612932911494, 3086.6548375322227, 1734.6819794769237, 1142.3905674171788, 889.6784854471445]
 Reconstructed Image Paths: {5: 'kmeans_compression_results\\l2\\football_k5_l2.png', 10: 'kmeans_compression_results\\l2\\football_k10_l2.png', 20: 'kmeans_compression_results\\l2\\football_k20_l2.png', 30: 'kmeans_compression_results\\l2\\football_k30_l2.png', 40: 'kmeans_compression_results\\l2\\football_k40_l2.png'}
 Image: flower
 K values: [5, 10, 20, 30, 40]
 Times (s): [0.5750138759613037, 0.9804377555847168, 1.8552260398864746, 2.748880624771118, 3.618640184402466]
 Iterations: [20, 20, 20, 20, 20]
 Distortions: [2576.560788507129, 1265.8500568542174, 723.535866552753, 470.35927640411245, 386.5890880381862]
 Reconstructed Image Paths: {5: 'kmeans_compression_results\\l2\\flower_k5_l2.png', 10: 'kmeans_compression_results\\l2\\flower_k10_l2.png', 20: 'kmeans_compression_results\\l2\\flower_k20_l2.png', 30: 'kmeans_compression_results\\l2\\flower_k30_l2.png', 40: 'kmeans_compression_results\\l2\\flower_k40_l2.png'}

Results for L1 norm:

Image: georgia-aquarium
 K values: [5, 10, 20, 30, 40]
 Times (s): [0.9835889339447021, 1.759988784790039, 3.301698684692383, 4.8300676345825195, 6.485980749130249]
 Iterations: [20, 20, 20, 20, 20]
 Distortions: [34541.0469838216, 24351.286924990145, 18963.300718366387, 15941.143461675805, 14564.418394362237]
 Reconstructed Image Paths: {5: 'kmeans_compression_results\\l1\\georgia-aquarium_k5_l1.png', 10: 'kmeans_compression_results\\l1\\georgia-aquarium_k10_l1.png', 20: 'kmeans_compression_results\\l1\\georgia-aquarium_k20_l1.png', 30: 'kmeans_compression_results\\l1\\georgia-aquarium_k30_l1.png', 40: 'kmeans_compression_results\\l1\\georgia-aquarium_k40_l1.png'}
 Image: football
 K values: [5, 10, 20, 30, 40]
 Times (s): [0.7919633388519287, 2.453557252883911, 4.7268383502960205, 6.80332088470459, 8.951244354248047]
 Iterations: [12, 20, 20, 20, 20]
 Distortions: [51820.22378688687, 38196.95444628665, 26244.21740559701, 22512.79786460296, 19731.274482793702]
 Reconstructed Image Paths: {5: 'kmeans_compression_results\\l1\\football_k5_l1.png', 10: 'kmeans_compression_results\\l1\\football_k10_l1.png', 20: 'kmeans_compression_results\\l1\\football_k20_l1.png', 30: 'kmeans_compression_results\\l1\\football_k30_l1.png', 40: 'kmeans_compression_results\\l1\\football_k40_l1.png'}

```

1.png', 10: 'kmeans_compression_results\\11\\football_k10_l1.png', 20: 'kmeans_compression_results\\11\\football_k20_l1.png', 30: 'kmeans_compression_results\\11\\football_k30_l1.png', 40: 'kmeans_compression_results\\11\\football_k40_l1.png'}
Image: flower
K values: [5, 10, 20, 30, 40]
Times (s): [0.6637334823608398, 1.075078010559082, 1.873084545135498, 2.767983913421631, 3.601811170578003]
Iterations: [20, 20, 20, 20, 20]
Distortions: [23172.71909877716, 15613.961188445744, 11381.731235530091, 9428.352575424427, 8463.855217108716]
Reconstructed Image Paths: {5: 'kmeans_compression_results\\11\\flower_k5_l1.png', 10: 'kmeans_compression_results\\11\\flower_k10_l1.png', 20: 'kmeans_compression_results\\11\\flower_k20_l1.png', 30: 'kmeans_compression_results\\11\\flower_k30_l1.png', 40: 'kmeans_compression_results\\11\\flower_k40_l1.png'}

--- End of Collected Results ---
Remember to populate your LaTeX tables with these values.

```

```

In [6]: # Cell 5: Optional - Visualize Reconstructed Images

print("\n--- Displaying Reconstructed Images ---")

for dist_type in norm_distances.keys():
    print(f"\nImages with {dist_type.upper()} Norm:")
    for image_name in image_paths.keys():
        print(f"\n {image_name} - {dist_type.upper()} Norm:")
        fig, axes = plt.subplots(1, len(k_values), figsize=(20, 4))
        fig.suptitle(f"{image_name} - {dist_type.upper()} Norm", fontsize=16)

        for i, k in enumerate(k_values):
            img_path = all_results[dist_type][image_name]['reconstructed_image_path']
            img = Image.open(img_path)
            axes[i].imshow(img)
            axes[i].set_title(f"k={k}")
            axes[i].axis('off')
        plt.tight_layout(rect=[0, 0.03, 1, 0.95]) # Adjust layout to prevent title
        plt.show()

print("\n--- End of Image Visualization ---")

--- Displaying Reconstructed Images ---

```

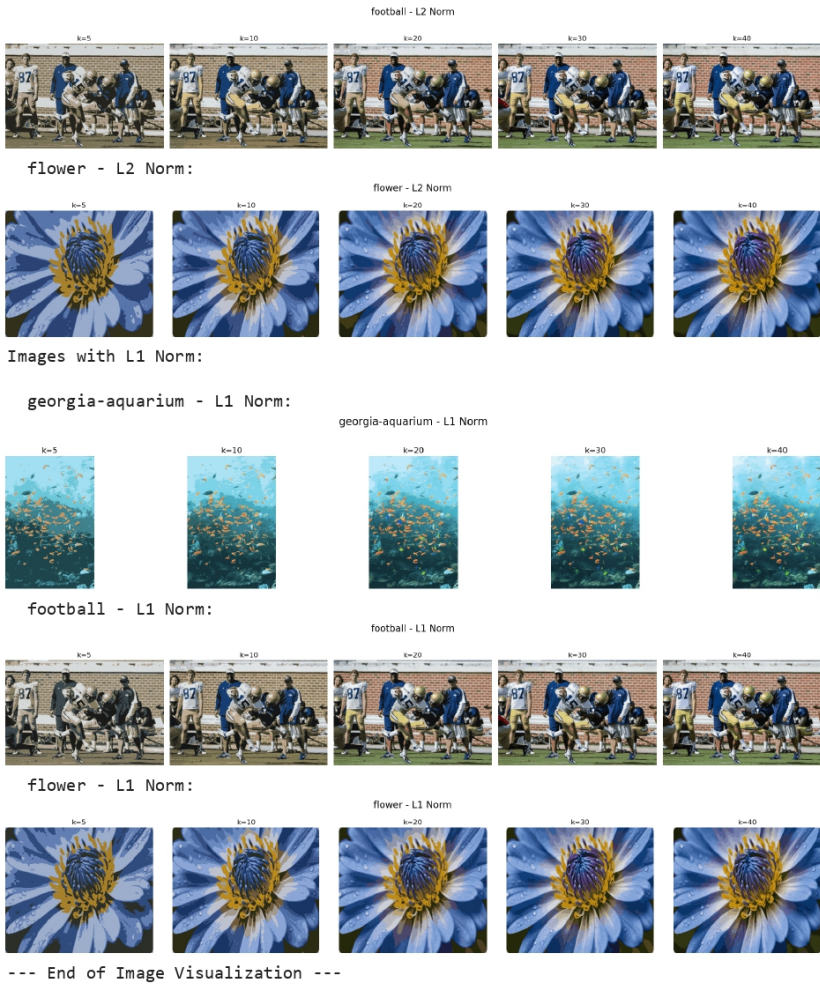
Images with L2 Norm:

georgia-aquarium - L2 Norm:

georgia-aquarium - L2 Norm



football - L2 Norm:



```
In [7]: # Cell 6: Plotting Elbow Graphs

print("\n--- Plotting Elbow Graphs ---")

# Ensure k_values are defined for plotting
# If you changed them in Cell 2, make sure this reflects it.
k_values = [5, 10, 20, 30, 40]

for dist_type, img_results in all_results.items():
    print(f"\nElbow Graphs for {dist_type.upper()} norm:")

    plt.figure(figsize=(15, 5))
    plt.suptitle(f"Elbow Method for K-Means Clustering ({dist_type.upper()} Norm)",

    for i, (image_name, data) in enumerate(img_results.items()):
        plt.subplot(1, len(img_results), i + 1)
        plt.plot(k_values, data['distortions'], marker='o', linestyle='-')
        plt.title(f"{image_name}")
        plt.xlabel("Number of Clusters (k)")
        plt.ylabel("Distortion")
        plt.grid(True)
        plt.xticks(k_values) # Ensure only tested k values are shown on x-axis

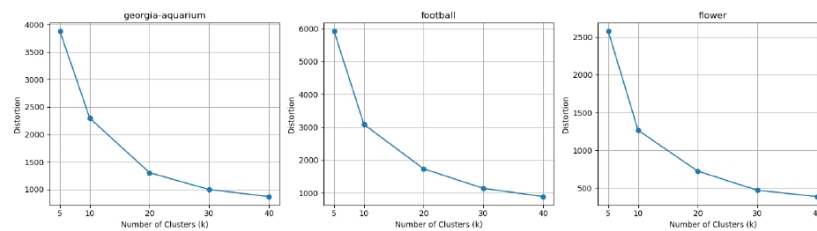
    plt.tight_layout(rect=[0, 0.03, 1, 0.95]) # Adjust layout to prevent title over
    plt.show()

print("\n--- Elbow Graph Plotting Complete ---")
print("Examine these plots to identify the 'elbow' point for each image and determi

--- Plotting Elbow Graphs ---
```

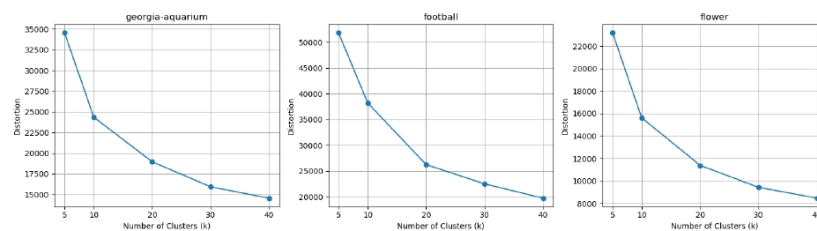
Elbow Graphs for L2 norm:

Elbow Method for K-Means Clustering (L2 Norm)



Elbow Graphs for L1 norm:

Elbow Method for K-Means Clustering (L1 Norm)



```
--- Elbow Graph Plotting Complete ---  
Examine these plots to identify the 'elbow' point for each image and determine the  
optimal k.
```

In []:

6.2 Political blogs dataset

Political Blog

<http://localhost:8888/nbconvert/html/Political%20Blog.ipynb?download...>

```
In [1]: # Cell 1: Import Libraries
import numpy as np
import pandas as pd
from collections import defaultdict
import matplotlib.pyplot as plt
from scipy.sparse import csgraph # Although csgraph is imported, it's not strictly

print("Libraries imported successfully.")

Libraries imported successfully.

In [2]: # Cell 2: Data Loading Function
def load_data(nodes_file, edges_file):
    """
    Loads node and edge data from files.

    Args:
        nodes_file (str): Path to the nodes.txt file.
        edges_file (str): Path to the edges.txt file.

    Returns:
        tuple: A tuple containing:
            - nodes_df (pd.DataFrame): DataFrame of nodes with 'id' and 'label'.
            - edges_df (pd.DataFrame): DataFrame of edges with 'source' and 'target'
    """
    # Load nodes data: id, blog_name, label, source_type
    nodes_df = pd.read_csv(nodes_file, sep='\t', header=None, names=['id', 'blog_na

    # Load edges data: source_id, target_id
    edges_df = pd.read_csv(edges_file, sep='\t', header=None, names=['source', 'tar

    return nodes_df, edges_df

print("load_data function defined.")

load_data function defined.
```

```

In [3]: # Cell 3: Graph Preprocessing Function
def preprocess_graph(nodes_df, edges_df):
    """
    Preprocesses the graph data: removes isolated nodes and creates adjacency matrix.

    Args:
        nodes_df (pd.DataFrame): DataFrame of nodes.
        edges_df (pd.DataFrame): DataFrame of edges.

    Returns:
        tuple: A tuple containing:
            - adj_matrix (np.ndarray): Symmetric adjacency matrix.
            - node_labels (np.ndarray): Labels of the remaining nodes, indexed by new_id.
            - id_to_new_id (dict): Mapping from original node ID to new contiguous ID.
            - new_id_to_id (dict): Mapping from new contiguous ID to original node ID.
    """
    # Get all unique nodes involved in edges
    # Using .values ensures it's a numpy array for unique()
    connected_nodes = np.unique(np.concatenate((edges_df['source'].values, edges_df['target'].values)))

    # Filter nodes_df to include only connected nodes
    filtered_nodes_df = nodes_df[nodes_df['id'].isin(connected_nodes)].copy()

    # Create a mapping from original node IDs to new contiguous IDs (0 to N-1)
    # This is crucial because original IDs might not be contiguous or start from 0
    id_to_new_id = {old_id: new_id for new_id, old_id in enumerate(filtered_nodes_df['id'])}
    new_id_to_id = {new_id: old_id for old_id, new_id in id_to_new_id.items()}

    num_nodes = len(id_to_new_id)

    # Initialize adjacency matrix with zeros
    adj_matrix = np.zeros((num_nodes, num_nodes), dtype=int)

    # Populate adjacency matrix
    for _, row in edges_df.iterrows():
        u_orig, v_orig = row['source'], row['target']

        # Only add edge if both nodes exist in the filtered set
        if u_orig in id_to_new_id and v_orig in id_to_new_id:
            u_new = id_to_new_id[u_orig]
            v_new = id_to_new_id[v_orig]

            # Since the graph is undirected, set both (u,v) and (v,u) to 1
            adj_matrix[u_new, v_new] = 1
            adj_matrix[v_new, u_new] = 1

    # Get labels for the filtered nodes, ordered by their new_id
    # Ensure labels are retrieved in the order of new_id_to_id keys
    node_labels = filtered_nodes_df.set_index('id').loc[list(new_id_to_id.values())].index.values

    return adj_matrix, node_labels, id_to_new_id, new_id_to_id

print("preprocess_graph function defined.")
preprocess_graph function defined.

```

```
In [4]: # Cell 4: Laplacian Computation Function
def compute_laplacian(adj_matrix):
    """
    Computes the unnormalized Laplacian matrix  $L = D - A$ .

    Args:
        adj_matrix (np.ndarray): The adjacency matrix.

    Returns:
        np.ndarray: The unnormalized Laplacian matrix.
    """
    degree_matrix = np.diag(np.sum(adj_matrix, axis=1))
    laplacian_matrix = degree_matrix - adj_matrix
    return laplacian_matrix

print("compute_laplacian function defined.")

compute_laplacian function defined.
```

```

In [5]: # Cell 5: K-Means Implementation
def euclidean_distance(point1, point2):
    """Calculates the Euclidean distance between two points."""
    return np.sqrt(np.sum((point1 - point2)**2))

def kmeans(data, k, max_iterations=100, tolerance=1e-4):
    """
    Performs K-Means clustering.

    Args:
        data (np.ndarray): The data points to cluster.
        k (int): The number of clusters.
        max_iterations (int): Maximum number of iterations for K-Means.
        tolerance (float): Convergence tolerance for centroid movement.

    Returns:
        np.ndarray: An array of cluster assignments for each data point.
    """
    num_samples, num_features = data.shape

    # 1. Initialize centroids randomly
    # Select k random data points as initial centroids
    random_indices = np.random.choice(num_samples, k, replace=False)
    centroids = data[random_indices]

    for iteration in range(max_iterations):
        # 2. Assign each data point to the closest centroid
        cluster_assignments = np.zeros(num_samples, dtype=int)
        for i in range(num_samples):
            distances = [euclidean_distance(data[i], centroid) for centroid in centroids]
            cluster_assignments[i] = np.argmin(distances)

        # 3. Update centroids based on the mean of assigned points
        new_centroids = np.zeros_like(centroids)
        for j in range(k):
            points_in_cluster = data[cluster_assignments == j]
            if len(points_in_cluster) > 0:
                new_centroids[j] = np.mean(points_in_cluster, axis=0)
            else:
                # Handle empty cluster: re-initialize centroid to a random point
                # This helps prevent issues if a cluster becomes empty
                new_centroids[j] = data[np.random.choice(num_samples)]

        # 4. Check for convergence
        centroid_movement = np.sum([euclidean_distance(centroids[j], new_centroids[j]) for j in range(k)])
        if centroid_movement < tolerance:
            # print(f"K-Means converged after {iteration + 1} iterations.") # Uncomment to see progress
            break

        centroids = new_centroids

    return cluster_assignments

print("kmeans and euclidean_distance functions defined.")
kmeans and euclidean_distance functions defined.

```

```

In [6]: # Cell 6: Spectral Clustering Function
def spectral_clustering(adj_matrix, k):
    """
    Performs spectral clustering on the given adjacency matrix.

    Args:
        adj_matrix (np.ndarray): The adjacency matrix.
        k (int): The number of clusters.

    Returns:
        np.ndarray: An array of cluster assignments for each node.
    """
    laplacian = compute_laplacian(adj_matrix)

    # Compute eigenvalues and eigenvectors
    # 'eigh' is used for symmetric matrices, which is more stable
    eigenvalues, eigenvectors = np.linalg.eigh(laplacian)

    # Sort eigenvalues and corresponding eigenvectors by eigenvalue magnitude
    sorted_indices = np.argsort(eigenvalues)
    eigenvalues = eigenvalues[sorted_indices]
    eigenvectors = eigenvectors[:, sorted_indices]

    # Select the k smallest non-zero eigenvectors
    # The first eigenvector (corresponding to eigenvalue 0) is trivial (all ones)
    # We look for eigenvalues slightly greater than zero due to numerical precision

    # Find indices of eigenvalues greater than a small tolerance
    non_zero_eigenvalue_indices = np.where(eigenvalues > 1e-8)[0]

    if len(non_zero_eigenvalue_indices) < k:
        print(f"Warning: Not enough non-zero eigenvalues for k={k}. Using all avail")
        selected_eigenvector_indices = non_zero_eigenvalue_indices
    else:
        # Select the first k non-zero eigenvectors
        selected_eigenvector_indices = non_zero_eigenvalue_indices[:k]

    # Construct the feature matrix U from the selected eigenvectors
    U = eigenvectors[:, selected_eigenvector_indices]

    # Normalize rows of U to unit length (important for K-Means)
    row_sums = np.linalg.norm(U, axis=1)
    # Handle rows with zero norm (e.g., if a node has no connections and its eigenv
    # Replace 0s in row_sums with 1s to avoid division by zero; these rows will rem
    row_sums[row_sums == 0] = 1
    U_normalized = U / row_sums[:, np.newaxis]

    # Perform K-Means clustering on the rows of U_normalized
    cluster_assignments = kmeans(U_normalized, k)

    return cluster_assignments

print("spectral_clustering function defined.")
spectral_clustering function defined.

```

```

In [7]: # Cell 7: Mismatch Rate Calculation Function
def calculate_mismatch_rate(cluster_assignments, node_labels, k):
    """
    Calculates the mismatch rate for each cluster.

    Args:
        cluster_assignments (np.ndarray): Array of cluster assignments for each node
        node_labels (np.ndarray): True labels of the nodes.
        k (int): Number of clusters.

    Returns:
        dict: A dictionary where keys are cluster IDs and values are their mismatch rates
    """
    mismatch_rates = {}
    for cluster_id in range(k):
        # Get true labels for nodes in the current cluster
        labels_in_cluster = node_labels[cluster_assignments == cluster_id]

        if len(labels_in_cluster) == 0:
            mismatch_rates[cluster_id] = 0.0 # No mismatch if cluster is empty
            continue

        # Count occurrences of each label (0 and 1)
        counts = defaultdict(int)
        for label in labels_in_cluster:
            counts[label] += 1

        # Determine the majority label
        # If counts are equal, default to 0 as majority (arbitrary tie-break)
        if 0 in counts and 1 in counts:
            majority_label = 0 if counts[0] >= counts[1] else 1
        elif 0 in counts: # Only 0s present
            majority_label = 0
        else: # Only 1s present (or empty, handled above)
            majority_label = 1

        # Calculate mismatches
        mismatches = sum(1 for label in labels_in_cluster if label != majority_label)

        # Calculate mismatch rate
        mismatch_rate = mismatches / len(labels_in_cluster)
        mismatch_rates[cluster_id] = mismatch_rate

    return mismatch_rates

print("calculate_mismatch_rate function defined.")
calculate_mismatch_rate function defined.

```

```

In [8]: # Cell 8: Main Execution - Data Loading and Preprocessing
nodes_file_path = 'nodes.txt'
edges_file_path = 'edges.txt'

# Load data
nodes_df, edges_df = load_data(nodes_file_path, edges_file_path)
print(f"Original nodes: {len(nodes_df)}")
print(f"Original edges: {len(edges_df)}")

# Preprocess graph (remove isolated nodes, create adjacency matrix)
adj_matrix, node_labels, id_to_new_id, new_id_to_id = preprocess_graph(nodes_df, ed
print(f"Nodes after removing isolated nodes: {adj_matrix.shape[0]}")
print(f"Adjacency matrix shape: {adj_matrix.shape}")

Original nodes: 1490
Original edges: 19090
Nodes after removing isolated nodes: 1224
Adjacency matrix shape: (1224, 1224)

In [9]: # Cell 9: Main Execution - Question 1: Spectral Clustering for k = 2, 5, 10, 30, 50
k_values = [2, 5, 10, 30, 50]
all_mismatch_rates = {}

print("\n--- Question 1 Results ---")
for k in k_values:
    print(f"\nClustering for k = {k}...")
    cluster_assignments = spectral_clustering(adj_matrix, k)
    mismatch_rates = calculate_mismatch_rate(cluster_assignments, node_labels, k)
    all_mismatch_rates[k] = mismatch_rates

    print(f"Mismatch rates for k = {k}:")
    for cluster_id, rate in mismatch_rates.items():
        print(f"  Cluster {cluster_id}: {rate:.4f}")

# Calculate average mismatch rate for this k
avg_mismatch_rate = np.mean(list(mismatch_rates.values()))
print(f"  Average Mismatch Rate for k = {k}: {avg_mismatch_rate:.4f}")

```

--- Question 1 Results ---

Clustering for k = 2...
Mismatch rates for k = 2:
Cluster 0: 0.2857
Cluster 1: 0.4826
Average Mismatch Rate for k = 2: 0.3842

Clustering for k = 5...
Mismatch rates for k = 5:
Cluster 0: 0.2963
Cluster 1: 0.4414
Cluster 2: 0.3600
Cluster 3: 0.0432
Cluster 4: 0.0340
Average Mismatch Rate for k = 5: 0.2350

Clustering for k = 10...
Mismatch rates for k = 10:
Cluster 0: 0.4706
Cluster 1: 0.1509
Cluster 2: 0.0185
Cluster 3: 0.0323
Cluster 4: 0.0231
Cluster 5: 0.2195
Cluster 6: 0.0370
Cluster 7: 0.3182
Cluster 8: 0.0417
Cluster 9: 0.0960
Average Mismatch Rate for k = 10: 0.1408

Clustering for k = 30...
Mismatch rates for k = 30:
Cluster 0: 0.0000
Cluster 1: 0.0465
Cluster 2: 0.0000
Cluster 3: 0.2979
Cluster 4: 0.2000
Cluster 5: 0.5000
Cluster 6: 0.0270
Cluster 7: 0.0000
Cluster 8: 0.0000
Cluster 9: 0.0238
Cluster 10: 0.0000
Cluster 11: 0.2414
Cluster 12: 0.2381
Cluster 13: 0.3333
Cluster 14: 0.0000
Cluster 15: 0.0000
Cluster 16: 0.0000
Cluster 17: 0.0769
Cluster 18: 0.0000
Cluster 19: 0.0417
Cluster 20: 0.0000
Cluster 21: 0.0317
Cluster 22: 0.0000
Cluster 23: 0.0000
Cluster 24: 0.0278
Cluster 25: 0.0000

```
Cluster 26: 0.0444
Cluster 27: 0.0000
Cluster 28: 0.0000
Cluster 29: 0.1111
Average Mismatch Rate for k = 30: 0.0747
```

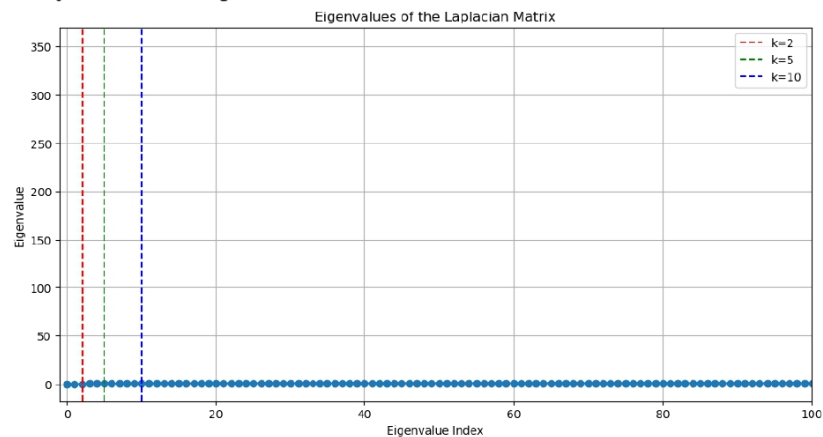
```
Clustering for k = 50...
Mismatch rates for k = 50:
Cluster 0: 0.0000
Cluster 1: 0.0245
Cluster 2: 0.1667
Cluster 3: 0.2000
Cluster 4: 0.0000
Cluster 5: 0.0000
Cluster 6: 0.3333
Cluster 7: 0.0000
Cluster 8: 0.0385
Cluster 9: 0.0000
Cluster 10: 0.0000
Cluster 11: 0.0345
Cluster 12: 0.2222
Cluster 13: 0.0000
Cluster 14: 0.0000
Cluster 15: 0.0000
Cluster 16: 0.1000
Cluster 17: 0.1000
Cluster 18: 0.0000
Cluster 19: 0.4545
Cluster 20: 0.3333
Cluster 21: 0.1765
Cluster 22: 0.0000
Cluster 23: 0.2500
Cluster 24: 0.0263
Cluster 25: 0.0000
Cluster 26: 0.0000
Cluster 27: 0.0769
Cluster 28: 0.0000
Cluster 29: 0.3333
Cluster 30: 0.0000
Cluster 31: 0.1429
Cluster 32: 0.0000
Cluster 33: 0.0000
Cluster 34: 0.0000
Cluster 35: 0.1111
Cluster 36: 0.0000
Cluster 37: 0.0000
Cluster 38: 0.0000
Cluster 39: 0.0000
Cluster 40: 0.0000
Cluster 41: 0.0000
Cluster 42: 0.0500
Cluster 43: 0.0000
Cluster 44: 0.0000
Cluster 45: 0.1538
Cluster 46: 0.0394
Cluster 47: 0.0000
Cluster 48: 0.0000
Cluster 49: 0.3243
Average Mismatch Rate for k = 50: 0.0738
```

```
In [10]: # Cell 10: Main Execution - Question 2: Tuning k - Eigenvalue Plot
print("\n--- Question 2: Tuning k ---")

# Plot eigenvalues to find a suitable k
laplacian = compute_laplacian(adj_matrix)
eigenvalues, _ = np.linalg.eigh(laplacian)
eigenvalues = np.sort(eigenvalues) # Ensure they are sorted

plt.figure(figsize=(12, 6))
plt.plot(range(len(eigenvalues)), eigenvalues, marker='o', linestyle='-')
plt.title('Eigenvalues of the Laplacian Matrix')
plt.xlabel('Eigenvalue Index')
plt.ylabel('Eigenvalue')
plt.grid(True)
plt.xlim(-1, 100) # Focus on the smaller eigenvalues for better visualization of po
plt.axvline(x=2, color='r', linestyle='--', label='k=2')
plt.axvline(x=5, color='g', linestyle='--', label='k=5')
plt.axvline(x=10, color='b', linestyle='--', label='k=10')
plt.legend()
plt.show()
```

--- Question 2: Tuning k ---



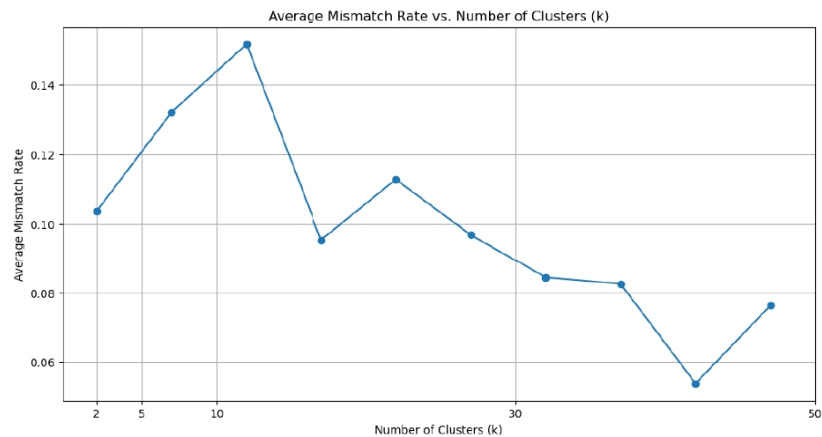
```
In [12]: # Cell 11: Main Execution - Question 2: Tuning k - Average Mismatch Rate Plot and C
# Re-run for a range of k values to find the best average mismatch rate
k_tuning_range = range(2, 51, 5) # Test k from 2 to 50
avg_mismatch_rates_tuning = []

for k_val in k_tuning_range:
    # Note: K-Means initialization is random, so results might vary slightly on eac
    # For robust tuning, one might run spectral_clustering multiple times for each
    # For this exercise, a single run per k_val is sufficient as per the prompt.
    cluster_assignments_tuned = spectral_clustering(adj_matrix, k_val)
    mismatch_rates_tuned = calculate_mismatch_rate(cluster_assignments_tuned, node_
    avg_mismatch_rates_tuning.append(np.mean(list(mismatch_rates_tuned.values()))))

plt.figure(figsize=(12, 6))
plt.plot(k_tuning_range, avg_mismatch_rates_tuning, marker='o', linestyle='-')
plt.title('Average Mismatch Rate vs. Number of Clusters (k)')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Average Mismatch Rate')
plt.grid(True)
plt.xticks(k_values) # Mark the specific k values from Q1 for reference
plt.show()

# Find the k with the minimum average mismatch rate
best_k = k_tuning_range[np.argmin(avg_mismatch_rates_tuning)]
min_mismatch_rate = np.min(avg_mismatch_rates_tuning)

print(f"\nOptimal k found by minimizing average mismatch rate: {best_k}")
print(f"Achieved minimum average mismatch rate: {min_mismatch_rate:.4f}")
```



Optimal k found by minimizing average mismatch rate: 42
Achieved minimum average mismatch rate: 0.0538

```
In [13]: # Optional: Display mismatch rates for the best_k if it wasn't already calculated i
if best_k not in k_values:
    print(f"\nClustering for optimal k = {best_k} (if not already calculated):")
    cluster_assignments_best_k = spectral_clustering(adj_matrix, best_k)
    mismatch_rates_best_k = calculate_mismatch_rate(cluster_assignments_best_k, nod
    for cluster_id, rate in mismatch_rates_best_k.items():
        print(f" Cluster {cluster_id}: {rate:.4f}")

Clustering for optimal k = 42 (if not already calculated):
Cluster 0: 0.2174
Cluster 1: 0.0000
Cluster 2: 0.0000
Cluster 3: 0.1111
Cluster 4: 0.0714
Cluster 5: 0.0000
Cluster 6: 0.0000
Cluster 7: 0.0000
Cluster 8: 0.3333
Cluster 9: 0.0323
Cluster 10: 0.0952
Cluster 11: 0.1250
Cluster 12: 0.0000
Cluster 13: 0.3158
Cluster 14: 0.0000
Cluster 15: 0.2414
Cluster 16: 0.0000
Cluster 17: 0.0000
Cluster 18: 0.1111
Cluster 19: 0.0455
Cluster 20: 0.0000
Cluster 21: 0.0000
Cluster 22: 0.0000
Cluster 23: 0.5000
Cluster 24: 0.0000
Cluster 25: 0.0000
Cluster 26: 0.0000
Cluster 27: 0.1250
Cluster 28: 0.1429
Cluster 29: 0.1579
Cluster 30: 0.4000
Cluster 31: 0.0526
Cluster 32: 0.5000
Cluster 33: 0.0000
Cluster 34: 0.0000
Cluster 35: 0.0000
Cluster 36: 0.0000
Cluster 37: 0.0000
Cluster 38: 0.0112
Cluster 39: 0.3333
Cluster 40: 0.0233
Cluster 41: 0.0000
```

In []:

6.3 MNIST Dataset clustering

```

In [1]: import numpy as np
        from collections import Counter
        from sklearn.cluster import KMeans
        from scipy.io import loadmat
        from scipy.spatial.distance import cdist

In [2]: data = loadmat("mnist_10digits.mat")
        xtrain = data['xtrain']
        ytrain = data['ytrain'].flatten()

        print("xtrain shape:", xtrain.shape)
        print("ytrain shape:", ytrain.shape)

xtrain shape: (60000, 784)
ytrain shape: (60000,)

In [3]: def compute_purity(labels, true_labels, n_clusters=10):
        purity_results = []
        for i in range(n_clusters):
            indices = np.where(labels == i)[0]
            if len(indices) == 0:
                purity_results.append({"true_label": None, "purity_score": 0})
                continue
            cluster_labels = true_labels[indices]
            most_common_label, count = Counter(cluster_labels).most_common(1)[0]
            purity = count / len(indices)
            purity_results.append({"true_label": int(most_common_label), "purity_score": purity})
        return purity_results

In [4]: from sklearn.cluster import KMeans

        x_norm = xtrain / 255.0
        kmeans = KMeans(n_clusters=10, random_state=42, n_init=10)
        cluster_labels_euclidean = kmeans.fit_predict(x_norm)

        purity_euclidean = compute_purity(cluster_labels_euclidean, ytrain)

        # Display results
        print("Purity using Euclidean Distance:")
        for i, result in enumerate(purity_euclidean):
            print(f"Cluster {i}: True Label = {result['true_label']}, Purity = {result['purity_score']}")

Purity using Euclidean Distance:
Cluster 0: True Label = 1, Purity = 0.5276
Cluster 1: True Label = 2, Purity = 0.8972
Cluster 2: True Label = 1, Purity = 0.6239
Cluster 3: True Label = 8, Purity = 0.5311
Cluster 4: True Label = 0, Purity = 0.9073
Cluster 5: True Label = 7, Purity = 0.4267
Cluster 6: True Label = 3, Purity = 0.5281
Cluster 7: True Label = 0, Purity = 0.7808
Cluster 8: True Label = 4, Purity = 0.3572
Cluster 9: True Label = 6, Purity = 0.8592

```

```

In [5]: def hamming_kmeans(X, k, max_iter=10):
    np.random.seed(42)
    centroids_idx = np.random.choice(X.shape[0], k, replace=False)
    centroids = X[centroids_idx]

    for iteration in range(max_iter):
        distances = cdist(X, centroids, metric='hamming')
        labels = np.argmin(distances, axis=1)

        new_centroids = np.zeros_like(centroids)
        for i in range(k):
            members = X[labels == i]
            if members.shape[0] > 0:
                new_centroids[i] = np.round(np.mean(members, axis=0))
            else:
                new_centroids[i] = centroids[i]
        centroids = new_centroids

    return labels, centroids

In [6]: # Thresholding the image pixels
x_binary = (xtrain > 128).astype(int)

# Apply Hamming-based k-means
cluster_labels_hamming, centroids_hamming = hamming_kmeans(x_binary, k=10)

purity_hamming = compute_purity(cluster_labels_hamming, ytrain)

# Display results
print("Purity using Hamming Distance:")
for i, result in enumerate(purity_hamming):
    print(f"Cluster {i}: True Label = {result['true_label']}, Purity = {result['pur"]

Purity using Hamming Distance:
Cluster 0: True Label = 5, Purity = 0.1873
Cluster 1: True Label = 3, Purity = 0.5637
Cluster 2: True Label = 8, Purity = 0.3676
Cluster 3: True Label = 1, Purity = 0.5421
Cluster 4: True Label = 0, Purity = 0.7663
Cluster 5: True Label = 6, Purity = 0.6325
Cluster 6: True Label = 7, Purity = 0.4960
Cluster 7: True Label = 1, Purity = 0.4177
Cluster 8: True Label = 0, Purity = 0.8743
Cluster 9: True Label = 4, Purity = 0.4675

In [7]: # Compare average purities
avg_purity_euclidean = np.mean([p['purity_score'] for p in purity_euclidean])
avg_purity_hamming = np.mean([p['purity_score'] for p in purity_hamming])

print(f"\nAverage Purity - Euclidean: {avg_purity_euclidean:.4f}")
print(f"Average Purity - Hamming : {avg_purity_hamming:.4f}")

if avg_purity_euclidean > avg_purity_hamming:
    print("Euclidean distance provides better clustering purity on MNIST.")
else:
    print("Hamming distance provides better clustering purity on MNIST.")

```

Untitled1

<http://localhost:8888/nbconvert/html/Desktop/Untitled1.ipynb?download...>

```
Average Purity - Euclidean: 0.6439
Average Purity - Hamming : 0.5315
Euclidean distance provides better clustering purity on MNIST.
```

In []: